Lab 3: 7-Segment Display Controller

Yuval Zecharia

10/10/2026

1. מטרת המעבדה

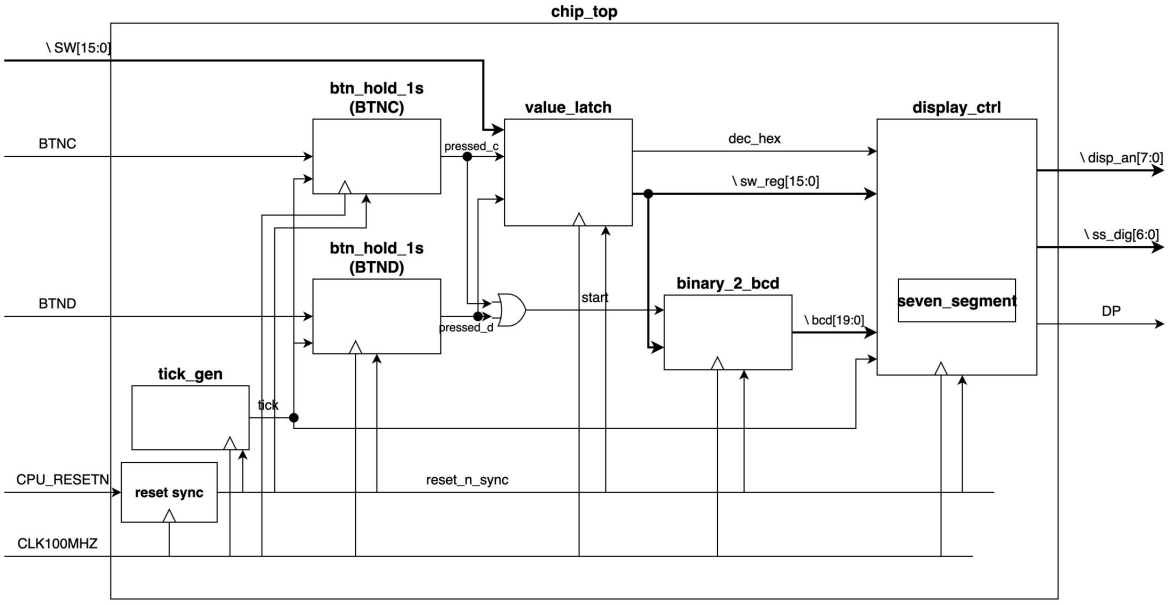
תכנון, מימוש ובדיקה של בקר לתצוגת שבעת המקטעים (7-Segment) של לוח ה-Nexys A7-100T, המציג את הערך הבינארי של 16 המתגים בייצוג עשרוני או הקסדצימלי. דרישות המעבדה:

1. הצגת הערך בעשרוני או בהקסדצימלי ללא הבהוב, בקצב רענון של 400Hz לכל ספרה, מתוך שעון מחולק של שעון המערכת (100MHz).
2. דגימת ערך המתגים לרגיסטר לאחר החזקה של שנייה אחת של הכפתור המרכזי (ייצוג עשרוני) או התחתון (ייצוג הקסדצימלי); כפתור ה-CPU RESET מאפס את הרגיסטר.
3. מפענח 7 מקטעים לכל ערך ספרה ובקר סריקה הקובע כמה ספרות נדרשות לערך המקסימלי.
4. טסטבנץ' לבקר התצוגה ולכל אבן בניין, עם timescale 1ns/1ps וזמן לחיצה מקוצר ל-100ns בסימולציה.
5. מימוש פיזי: הגדרת המתגים, הכפתורים, האנודות והקתודות בקובץ ה-XDC, סינתזה, Place & Route, יצירת bitstream ובדיקה על הלוח.

הדוח מרכז את תהליך התכנון, קוד המקור והטסטבנצ'ים, תוצאות הסימולציה, דוחות ה-Lint, הסינתזה והמימוש, ניצול המשאבים ובדיקת ה-bitstream על הלוח.

2. תהליך התכנון

2.1. ארכיטקטורה



איור 1: מבנה chip_top: תת-המודולים והאותות ביניהם.

המערכת מקבלת את ערך 16 המתגים SW[15:0] ומציגה אותו על תצוגת שבעת המקטעים של לוח ה-Nexys A7-100T בייצוג עשרוני (עד 5 ספרות, 65535) או הקסדצימלי (4 ספרות, FFFF). הערך אינו מוצג באופן רציף אלא נדגם לרגיסטר רק לאחר החזקת הכפתור המרכזי למשך שנייה אחת (BTNC – ייצוג עשרוני) או הכפתור התחתון (BTND – ייצוג הקסדצימלי). כפתור ה-CPU RESET (פעיל בנמוך) מאפס את הרגיסטר ומחזיר את התצוגה ל-0 בייצוג הקסדצימלי. כל המודולים פועלים על שעון יחיד של 100MHz, וכל האירועים האיטיים מיוצגים על ידי פולסים של מחזור שעון אחד (tick) המשמשים כ-enable סינכרוני ולא כשעון מחולק.

טבלה 1: תת-המודולים של העיצוב ותפקידם.

Module	File	Function
chip_top	src/chip_top.v	Top level: 2-FF synchronizer for CPU_RESETN, all instances, start = pressed_c | pressed_d
tick_gen	src/tick_gen.v	Divides the 100 MHz clock into a one-clock tick every MAX_TICK cycles (2 kHz)
btn_hold_1s (x2)	src/btn_hold_1s.v	Synchronizes a push-button and gives one pulse after it was held for MAX_COUNT ticks (1 s)
value_latch	src/value_latch.v	Latches SW on a button pulse and stores the presentation mode (dec_hex)
binary_2_bcd	src/binary_2_bcd.v	Serial double-dabble converter: 16-bit binary to 5 BCD digits in 16 clocks
display_ctrl	src/display_ctrl.v	Scans the 5 digits at 400 Hz: anode ring, nibble select, 5th digit only in decimal
seven_segment	src/seven_segment.v	Combinational hex digit to 7 active-low cathodes

2.2. ממשקי המודולים

להלן פורטי כל מודול, כיוונם, סוגם ותפקידם, והפרמטרים של chip_top.

טבלה 2: ממשק chip_top.

Port name	Direction	Type	Description
CLK100MHZ	Input	Wire	100 MHz system clock from the board oscillator
CPU_RESETN	Input	Wire	CPU reset push-button, active low; synchronized inside with 2 FFs
SW	Input	Wire [15:0]	16 slide switches – the binary value to display
BTNC	Input	Wire	Center push-button: 1 s hold latches SW and selects decimal
BTND	Input	Wire	Down push-button: 1 s hold latches SW and selects hexadecimal
DP	Output	Wire	Decimal point cathode (active low) – constant 1 (off)
ss_dig	Output	Wire [6:0]	7-segment cathodes {CA..CG} of the active digit, active low
disp_an	Output	Wire [7:0]	Digit anodes AN7..AN0, active low, one digit on at a time

טבלה 3: הפרמטרים של chip_top וערכיהם בחומרה ובסימולציה.

Parameter	Hardware value	Simulation (tb_chip_top)	Meaning
MAX_TICK	50000	2	tick_gen divider for the 2 kHz tick (0.5 ms) – equation (1)
MAX_COUNT	2000	5	btn_hold_1s: ticks counted for a 1 s press – equation (5)

טבלה 4: ממשק tick_gen.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronized reset from chip (Active low)
tick	Output	Wire	One-clock pulse every MAX_TICK clocks (2 kHz); also clears the counter

טבלה 5: ממשק btn_hold_1s.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronized reset from chip (Active low)
click	Input	Wire	Raw push-button (asynchronous), 1 while pressed
tick	Input	Wire	2 kHz tick from tick_gen
pressed_1s	Output	Wire	One-clock pulse after the button was held for MAX_COUNT ticks (1 s); one pulse per press

טבלה 6: ממשק value_latch.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronized reset from chip (Active low)
pressed_c	Input	Wire	1 s pulse of the center button
pressed_d	Input	Wire	1 s pulse of the down button
sw	Input	Wire [15:0]	Switch value to latch
dec_hex	Output	Reg	1 = decimal presentation (center), 0 = hexadecimal (down or reset)
sw_reg	Output	Reg [15:0]	Latched switch value (0 after reset)

טבלה 7: ממשק binary_2_bcd.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronized reset from chip (Active low)
sw	Input	Wire [15:0]	Latched binary value (sw_reg)
start	Input	Wire	Clears the converter and starts 16 shift cycles
bcd	Output	Wire [19:0]	5 BCD digits {ten-thousands..ones}, valid 16 clocks after start

טבלה 8: ממשק display_ctrl.

Port name	Direction	Type	Description
clk	Input	Wire	Clock input from chip
reset_n	Input	Wire	Synchronized reset from chip (Active low)
tick	Input	Wire	2 kHz scan tick – advances to the next digit
dec_hex	Input	Wire	Presentation select: 1 = decimal (5 digits), 0 = hex (4 digits)
hex_val	Input	Wire [15:0]	Value for the hexadecimal presentation
bcd_val	Input	Wire [19:0]	Value for the decimal presentation
display_an	Output	Wire [7:0]	Active-low anode select, one digit per tick; AN4 only in decimal, AN7..AN5 always off
ss_dig	Output	Wire [6:0]	Cathodes of the digit currently selected
dp	Output	Wire	Decimal point – constant 1 (off)

טבלה 9: ממשק seven_segment.

Port name	Direction	Type	Description
dig_in	Input	Wire [3:0]	Hex digit 0–F to display
ss_dig	Output	Reg [6:0]	Active-low cathodes {CA,CB,CC,CD,CE,CF,CG}

2.3. שיקולי תכנון

- מספר ספרות: הערך המקסימלי 65535 דורש 5 ספרות בייצוג עשרוני ו-4 ספרות בייצוג הקסדצימלי. לכן נעשה שימוש ב-5 אנודות (AN0–AN4), כאשר AN4 מודלקת רק במצב עשרוני; AN5–AN7 ו-DP כבויים תמיד.
- קצב רענון: tick_gen מחלק את שעון ה-100MHz ב-MAX_TICK ומפיק פולס tick בתדר 2kHz, כלומר כל 0.5ms (משוואה 1); רוחב המונה נגזר ממשוואה 2. בכל tick מתקדם display_ctrl לספרה הבאה, ועם 5 מקומות כל ספרה מודלקת פעם ב-2.5ms – קצב רענון של 400Hz לכל ספרה ללא הבהוב (משוואה 3).

$$MAX_TICK = \frac{f_{clk}}{f_{tick}} = f_{clk} \cdot T_{tick} = 100\ MHz \cdot 0.5\ ms = 50000 \quad (1)$$

$$N_{counter} = \lceil log_2(MAX_TICK) \rceil = \lceil log_2 50000 \rceil = 16\ bit \quad (2)$$

$$f_{refresh} = \frac{f_{tick}}{N_{digits}} = \frac{2\ kHz}{5} = 400\ Hz,\ T_{refresh} = 5 \cdot T_{tick} = 2.5\ ms \quad (3)$$

- פולס של 1Hz: ניתן ליצור פולס 1Hz ישירות משעון המערכת, אך הדבר דורש מונה של 27 ביט (משוואה 4). במקום זאת btn_hold_1s סופר MAX_COUNT פולסים של ה-tick במונה של 11 ביט (משוואה 5). בשני המקרים אין צורך בשעון מחולק: ה-tick משמש כ-enable סינכרוני בתחום שעון יחיד, ולכן אין חציית תחומי שעון, אין BUFG נוסף ואין אילוצי תזמון נוספים.

$$N_{1Hz} = f_{clk} \cdot 1\ s = 10^8 \ \Rightarrow \lceil log_2 10^8 \rceil = 27\ bit \quad (4)$$

$$MAX_COUNT = \frac{T_{hold}}{T_{tick}} = \frac{1\ s}{0.5\ ms} = 2000 \ \Rightarrow \lceil log_2 2000 \rceil = 11\ bit \quad (5)$$

- סנכרון: הכפתורים ו-CPU_RESETN הם אותות אסינכרוניים ולכן עוברים סנכרון בשני דלגלגים. מתגי ה-SW נדגמים רק ברגע הפולס של הכפתור ולכן אינם דורשים סנכרון.
- המרה לעשרוני: binary_2_bcd מממש את אלגוריתם Double Dabble (shift-and-add-3) באופן סדרתי – 16 מחזורי שעון לאחר start (משוואה 6). זמן זה זניח ביחס ל-tick של 0.5ms, ולכן התצוגה אינה מציגה לעולם תוצאה חלקית.

$$T_{conv} = 16 \cdot T_{clk} = 16 \cdot 10\ ns = 160\ ns \quad (6)$$

- קוטביות הלוח: האנודות והקתודות פעילות בנמוך ('0' מדליק), CPU_RESETN פעיל בנמוך, הכפתורים '1' בזמן לחיצה, ומתג במצב עליון = '1'. הפינים הוגדרו בקובץ ה-XDC של הלוח (מתגים, כפתורים, אנודות, קתודות ו-DP).
- סימולציה: זמן הלחיצה מקוצר ל-100ns על ידי דריסת הפרמטרים בטסטבנץ' בלבד (tb_chip_top: MAX_TICK=2, MAX_COUNT=5). ערכי החומרה בקוד (50000, 2000) נשארים ללא שינוי, ולכן אין צורך להחזירם לפני הסינתזה.

2.4. קוד המקור

קוד המקור של שבעת המודולים (תיקיית src: chip_top.v, tick_gen.v, btn_hold_1s.v, value_latch.v, binary_2_bcd.v, display_ctrl.v, seven_segment.v) וקטע האילוצים הפעיל מקובץ ה-XDC של הלוח מובאים בנספח א'. כל קובץ פותח ב-timescale 1ns/1ps.

2.5. סביבת הבדיקה

לכל מודול נכתב טסטבנץ' הבודק את עצמו: הטסטבנץ' מפיק גירויים, מחשב את התוצאה הצפויה במודל התנהגותי משלו (למשל פענוח תבנית הקתודות חזרה לספרה או חישוב BCD בחילוק), מדפיס שורת

PASS/FAIL לכל בדיקה ומסיים בשורת TEST PASS/FAIL. כל הקבצים משתמשים ב-timescale 1ps/1ns ובשעון של 10ns; זמן הלחיצה מקוצר ל-100ns בדריסת פרמטרים בלבד.

טבלה 10: הטסטבנצ'ים ומה שכל אחד מהם בודק.

Testbench	DUT	What is checked	Checks
tb_ss	seven_segment	all 16 digits: the cathode pattern decoded back equals the input	16
tb_tick_gen	tick_gen (MAX_TICK=5)	every tick interval = MAX_TICK clocks, tick is one clock wide	19
tb_value_latch	value_latch	reset values, no latch without a pulse, center / down latch and mode, hold, both pulses (down wins)	6
tb_btn_hold_1s	btn_hold_1s (MAX_COUNT=2)	short presses give no pulse, a hold gives exactly one pulse after MAX_COUNT ticks (latency window), all tick phases, bounce	18
tb_binary_2_bcd	binary_2_bcd	start clears; 0, 65535, 12345, 67890 (wraps to 2354) and 20 random values against a division model	25
tb_display_ctrl	display_ctrl + seven_segment	anode sequence in hex (AN4 never on) and decimal (AN4 on), digit at every position, mode switch mid-scan, 40 random values, DP off	525
tb_chip_top	chip_top (MAX_TICK=2, MAX_COUNT=5)	system: digit shown on every anode after each scenario – reset, 12345 dec, hold without press, 777 hex, short press, 65535/FFFF, 0, ABCD both buttons, 4321, reset, 10 random values	189

קוד הטסטבנצ'ים (תיקיית sim) מובא בנספח ב'; פלט ההרצה של כל טסטבנץ' מוצג בפרק 4 לצד צורות הגל.

3. בדיקת Lint

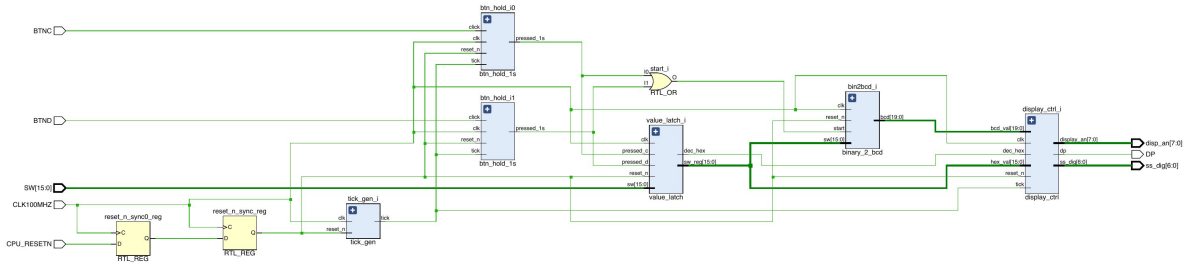
ה-Linter (RTL ANALYSIS → Run Linter) הורץ על chip_top יחד עם כל תתי-המודולים ולא מצא הפרות. ב-Elaborated Design דוחות Report DRC ו-Report Methodology נקיים (0 checks).

טבלה 11: תוצאות בדיקות ה-Lint וה-Elaborated Design.

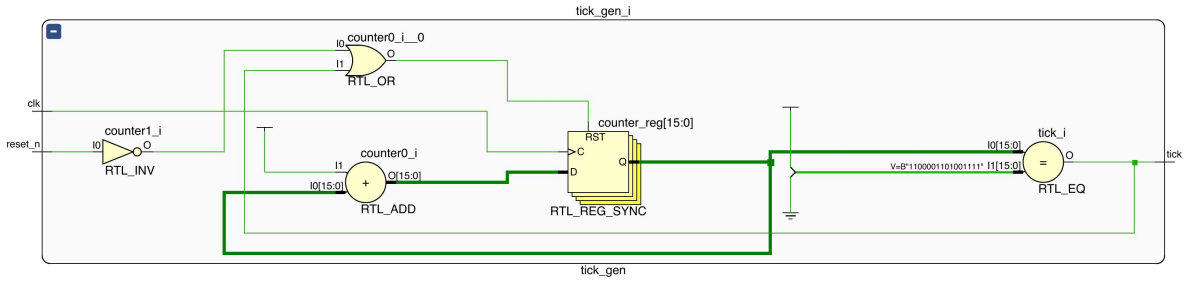
Check	Scope	Result	Report file
RTL Linter	chip_top + all sub-modules	0 violations, 0 waived	2_Lint_and_RTL_Schematics/vivado_lint_report_chip_top_with_all_submodules.txt
Elaborated design – Report DRC	chip_top (169 instances)	0 checks	elaborated_design_drc.rpt
Elaborated design – Report Methodology	chip_top	0 checks	elaborated_design_methodology.rpt

3.1. סכמות RTL (Elaborated Design)

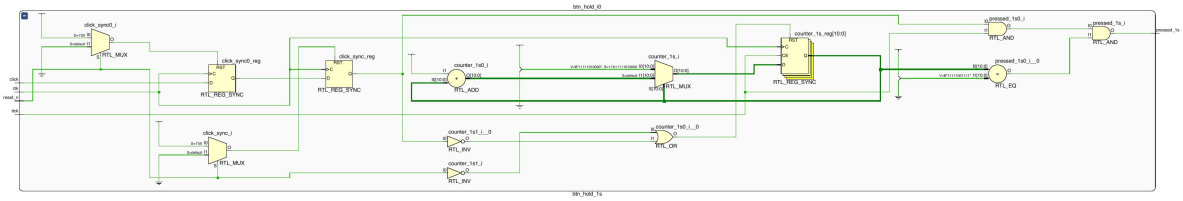
סכמות ה-elaboration של כל המודולים כפי שהן מוצגות ב-RTL ANALYSIS → Schematic:



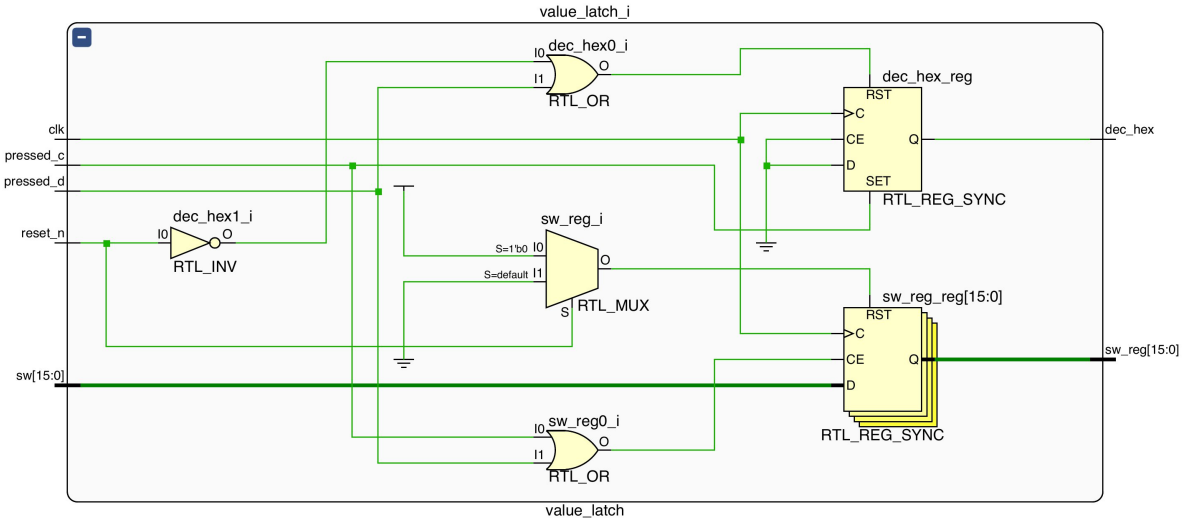
איור 2: סכמת ה-RTL של chip_top: סנכרון ה-reset, tick_gen, שני מופעי btn_hold_1s, value_latch, binary_2_bcd ו-display_ctrl.



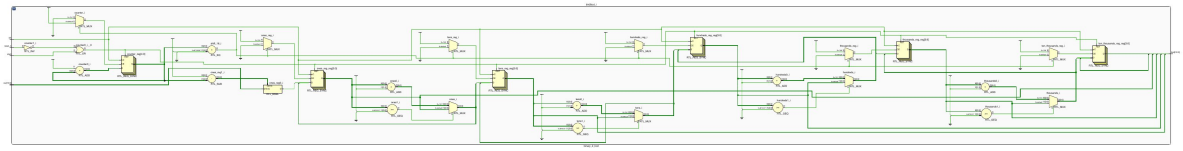
איור 3: סכמת ה-RTL של tick_gen: מונה 16 ביט, משווה ואיפוס סינכרוני.



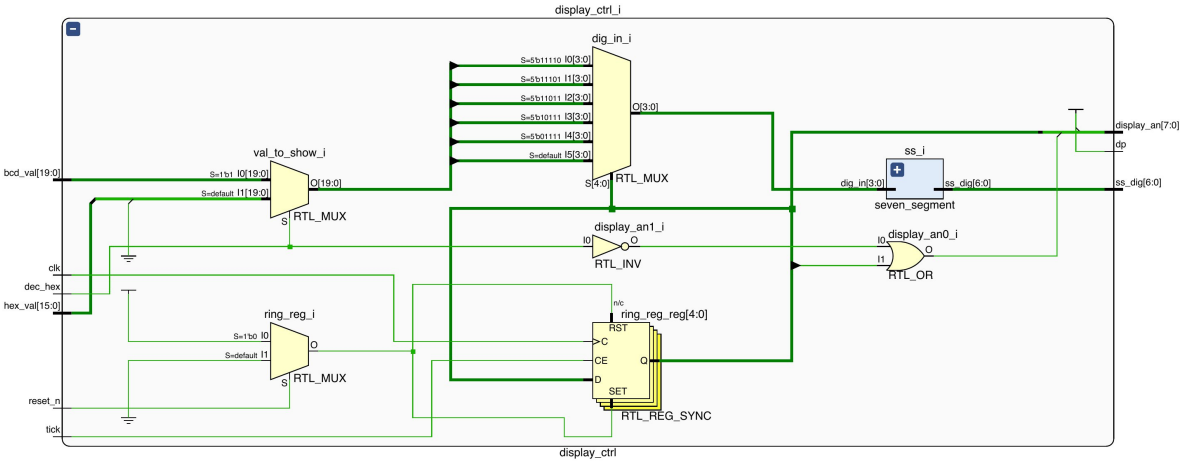
איור 4: סכמת ה-RTL של btn_hold_1s: סנכרונייזר, מונה ticks עם רוויה, משווה ו-AND ביציאה.



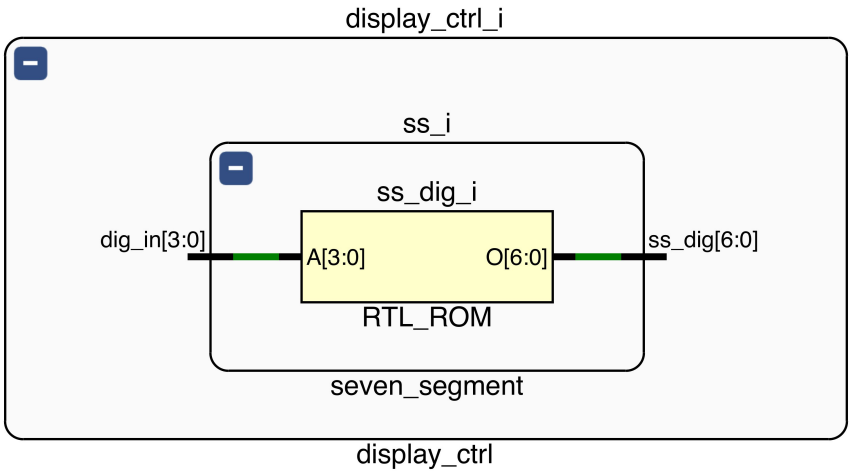
איור 5: סכמת ה-RTL של value_latch: רגיסטר 16 ביט עם clock enable ודלגלג המצב dec_hex.



איור 6: סכמת ה-RTL של binary_2_bcd: מונה המחזורים, בורר הביטים, שלבי ה-add-3 ורגיסטרי הספרות.



איור 7: סכמת ה-RTL של display_ctrl: רגיסטר הטבעת, בורר הערך והספרה, לוגיקת האנודות ומופע seven_segment.



איור 8: סכמת ה-RTL של seven_segment: משפט ה-case כמפענח בסגנון ROM.

4. תוצאות סימולציה

התמונות הן צילומי מסך של חלון ה-Waveform: לכל טסטבנץ' מוצגות הגדלות (zoom) של הקטעים החשובים, עם סמנים (markers) ממוספרים המצביעים על האירועים המוסברים בטקסט שאחרי כל איור. לפני האיורים מובא פלט הטסטבנץ' מה-Tcl console (שורות ה-PASS/FAIL ושורת הסיכום).

טבלה 12: סיכום תוצאות הסימולציה.

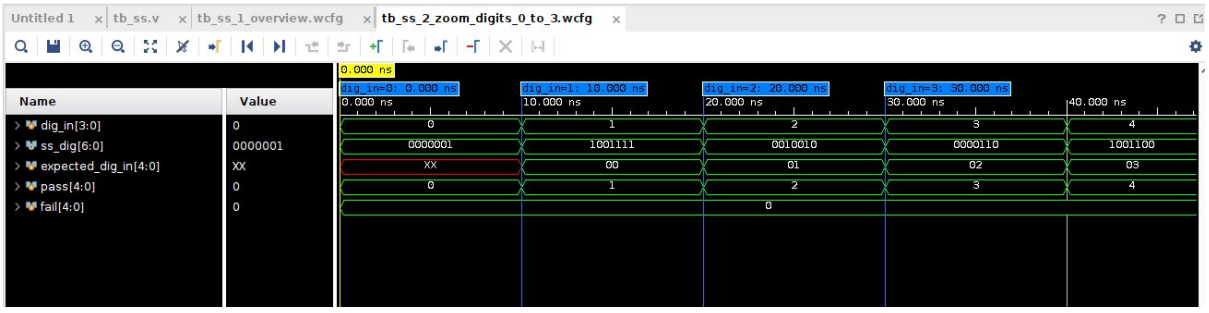
Testbench	DUT	Checks	Result	Sim time
tb_ss	seven_segment	16/16	TEST PASS	170 ns
tb_tick_gen	tick_gen (MAX_TICK=5)	19/19	TEST PASS	1020 ns
tb_value_latch	value_latch	6/6	TEST PASS	66 ns
tb_btn_hold_1s	btn_hold_1s (MAX_COUNT=2)	18/18	TEST PASS	2106 ns
tb_binary_2_bcd	binary_2_bcd	25/25	TEST PASS	4390 ns
tb_display_ctrl	display_ctrl + seven_segment	525/525	TEST PASS	8520 ns
tb_chip_top	chip_top (MAX_TICK=2, MAX_COUNT=5)	189/189	TEST PASS	11446 ns
Total	7 testbenches	798/798	0 failures	

4.1. tb_ss – בדיקת seven_segment

פלט הטסטבנץ' (tb_ss_xsim.log):

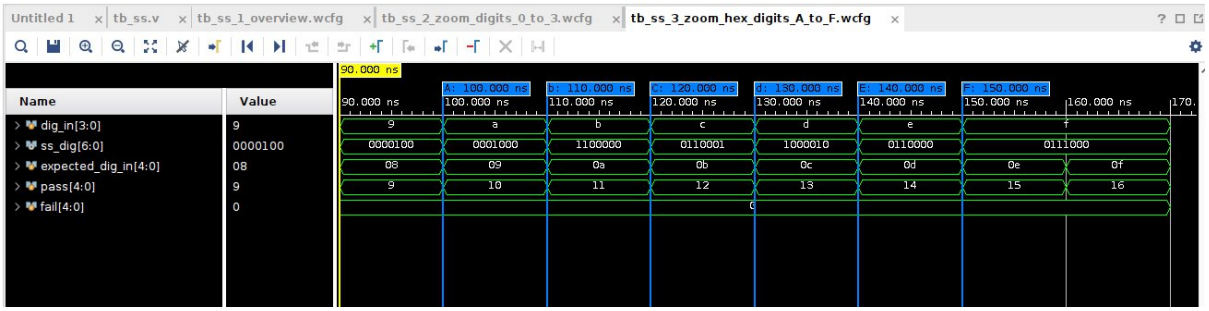
```
Time:  10 | digit:0 | decoded:0 | ss:0000001 | PASS
Time:  20 | digit:1 | decoded:1 | ss:1001111 | PASS
Time:  30 | digit:2 | decoded:2 | ss:0010010 | PASS
Time:  40 | digit:3 | decoded:3 | ss:0000110 | PASS
  ... (12 more checks) ...
Time: 170 | TEST PASS 16/16
```

dig_in עובר על 0–F כל 10ns; ss_dig מציג את תבנית הקתודות (פעיל בנמוך) ו-expected_dig_in הוא הספרה שהטסטבנץ' פענח מהקתודות.



איור 9: tb_ss – זום: הספרות 0 עד 3.

זום על הספרות 0 = 0000001 :0–3 (רק G כבוי), 1 = 1001111, 2 = 0010010, 3 = 0000110.



איור 10: tb_ss – זום: הספרות ההקסדצימליות A עד F.

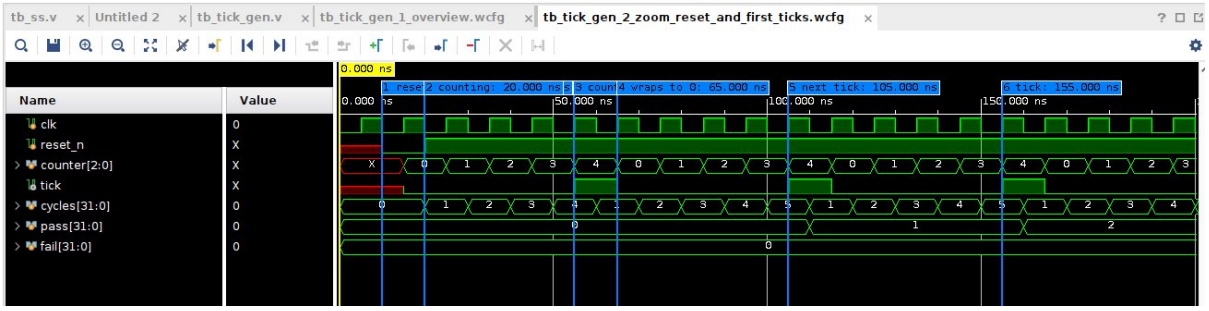
זום על הספרות ההקסדצימליות A–F (b) ו-d קטנות כדי להבדילן מ-8 ו-0).

4.2. tb_tick_gen – בדיקת tick_gen

פלט הטסטבנץ' (tb_tick_gen_xsim.log):

```
Time:  110 | Check tick interval correct | actual=     5 | expected=     5 | PASS
Time:  160 | Check tick interval correct | actual=     5 | expected=     5 | PASS
Time:  210 | Check tick interval correct | actual=     5 | expected=     5 | PASS
Time:  260 | Check tick interval correct | actual=     5 | expected=     5 | PASS
  ... (15 more checks) ...
Time: 1020 | TEST PASS | 19/19
```

MAX_TICK=5: לאחר שחרור ה-reset מתקבל tick כל 50ns, כלומר 5 מחזורי שעון; נמדדו 19 מרווחים רצופים, כולם שווים ל-5.



איור 11: tb_tick_gen – זום: reset וה-ticks הראשונים.

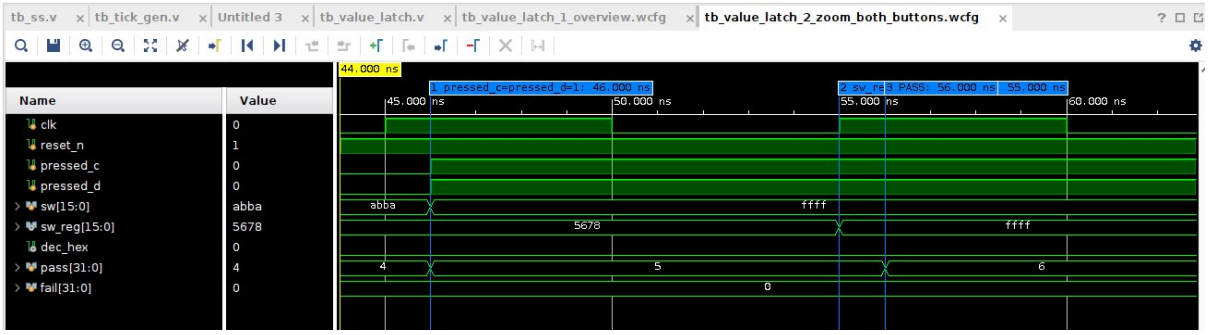
זום: המונה מוחזק ב-0 בזמן reset (1), סופר 0–4 לאחר השחרור (2), tick פעיל כשהמונה שווה 4 (3) ומאפס אותו במחזור הבא (4); המחזור הוא בדיוק 5 שעונים (5, 6).

4.3. tb_value_latch – בדיקת value_latch

פלט הטסטבנץ' (tb_value_latch_xsim.log):

```
Time:   6 | dec_hex=0 exp=0 | sw_reg=0000 exp=0000 | PASS
Time:  16 | dec_hex=0 exp=0 | sw_reg=0000 exp=0000 | PASS
Time:  26 | dec_hex=1 exp=1 | sw_reg=1234 exp=1234 | PASS
Time:  36 | dec_hex=0 exp=0 | sw_reg=5678 exp=5678 | PASS
  ... (2 more checks) ...
Time:  66 | TEST PASS | 6/6
```

לאחר reset: sw_reg=0 ומצב hex; pressed_c דוגם 1234 ובוחר עשרוני; pressed_d דוגם 5678 ובוחר hex; SW=ABBA ללא פולס אינו נדגם; שני הפולסים יחד דוגמים FFFF ו-down גובר – hex; 6/6 בדיקות עוברות.



איור 12: tb_value_latch – זום: לחיצה על שני הכפתורים יחד.

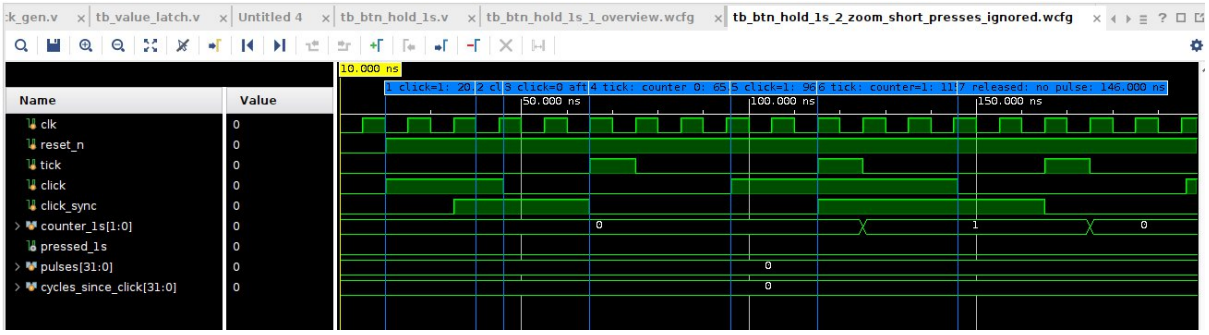
זום על לחיצה על שני הכפתורים יחד (1): בעליית השעון שאחרי pressed_c=pressed_d=1 הערך נדגם ו-dec_hex נשאר (2) 0; הבדיקה עוברת (3).

4.4. tb_btn_hold_1s – בדיקת btn_hold_1s

פלט הטסטבנץ' (tb_btn_hold_1s_xsim.log):

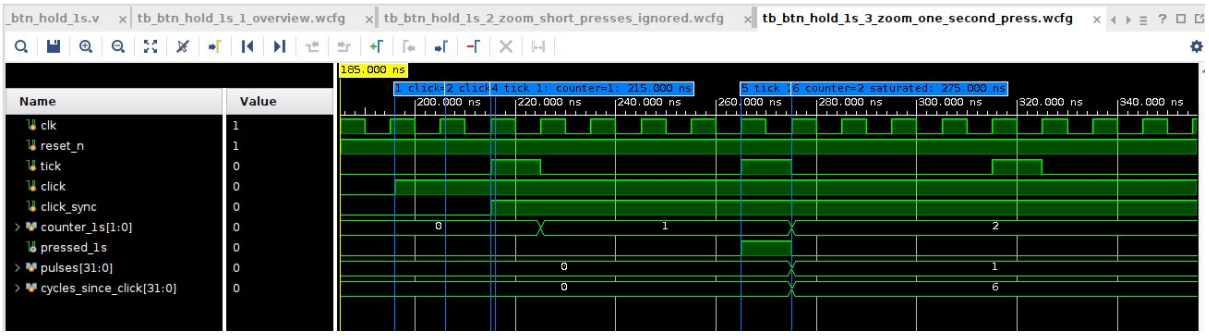
```
Time:   96 | Check if pressed_1s pulsed only once | actual=0 | expected=0 | PASS
Time:  196 | Check if pressed_1s pulsed only once | actual=0 | expected=0 | PASS
Time:  746 | Check press_1s goes high after +-1sec | actual=6 | min=6 | max=10 | PASS
Time:  746 | Check if pressed_1s pulsed only once | actual=1 | expected=1 | PASS
   ... (14 more checks) ...
Time: 2106 | TEST PASS | 18/18
```

MAX_COUNT=2 ticks, tick כל 5 שעונים: לחיצות של 3 ו-5 מחזורים אינן מייצרות פולס; החזקה ארוכה מייצרת פולס אחד לאחר 2 ticks וללא פולס נוסף; לחיצה של "שנייה" בכל אחת מ-6 הפאזות של ה-tick מייצרת פולס אחד בדיוק; ארבעה ריצודים של מחזור אחד מתעלמים והלחיצה היציבה שאחריהם מזוהה; 18/18 בדיקות עוברות.



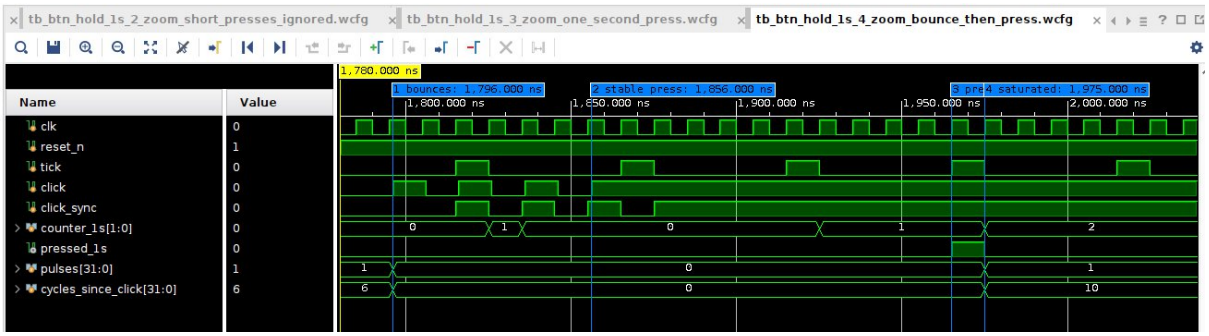
איור 13: tb_btn_hold_1s – זום: לחיצות קצרות מתעלמות.

click עולה (1) ומסונכרן אחרי שני שעונים (2); לחיצה של 3 מחזורים משתחררת (3) לפני ה-(4) tick, ולכן המונה נשאר 0; לחיצה של 5 מחזורים (5) מגיעה ל-counter_1s=1 ב-tick הראשון (6) אך משתחררת לפני ה-tick השני – המונה מתאפס ואין פולס (7).



איור 14: tb_btn_hold_1s – זום: לחיצה של שנייה.

click עולה click_sync0 (1), ו-(2, 3) click_sync, ה-tick הראשון מקדם את counter_1s ל-(4) 1, ה-tick השני עם counter_1s==MAX_COUNT-1 מייצר פולס pressed_1s של מחזור אחד (5), ואז המונה נעצר ב-2 כך שאין פולס נוסף (6).



איור 15: tb_btn_hold_1s – זום: ריצוד ואחריו לחיצה יציבה.

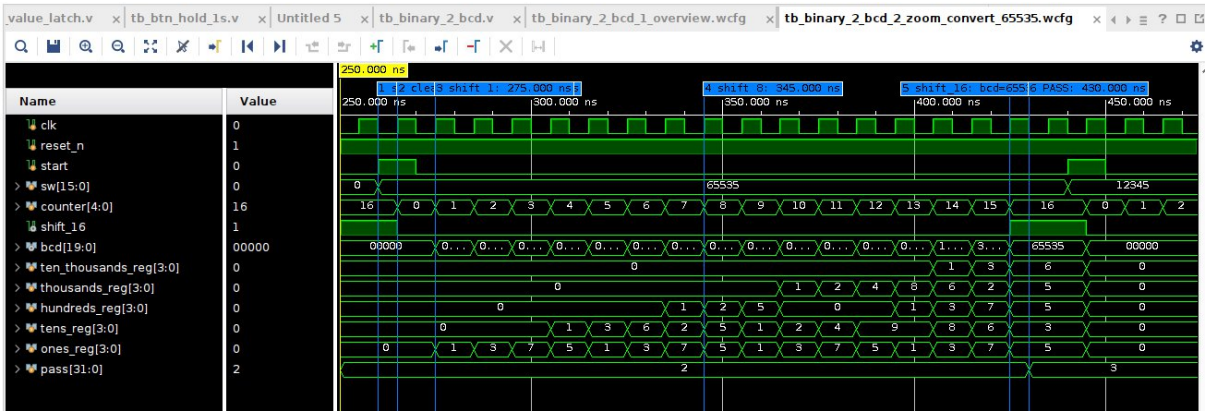
מעברים של מחזור אחד (1) מאפסים את המונה בכל פעם; הלחיצה היציבה שאחריהם (2) מזוהה לאחר שני ticks (3) והמונה נעצר (4).

4.5. tb_binary_2_bcd – בדיקת binary_2_bcd

פלט הטסטבנץ' (tb_binary_2_bcd_xsim.log):

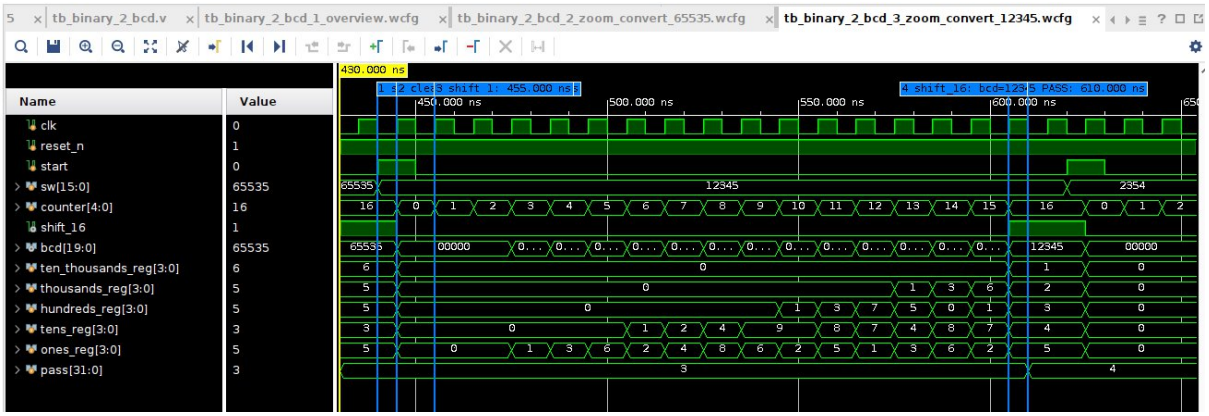
```
Time:   66 | Check BCD conversion correct | sw=0011000000111001 | actual_bcd=0000000000000000000 |
expected_bcd=0000000000000000000 | PASS
Time:  250 | Check BCD conversion correct | sw=0000000000000000 | actual_bcd=0000000000000000000 |
expected_bcd=0000000000000000000 | PASS
Time:  430 | Check BCD conversion correct | sw=1111111111111111 | actual_bcd=0110010101010100110101 |
expected_bcd=0110010101010100110101 | PASS
Time:  610 | Check BCD conversion correct | sw=0011000000111001 | actual_bcd=0001001000110100101 |
expected_bcd=0001001000110100101 | PASS
   ... (21 more checks) ...
Time:  4390 | TEST PASS | 25/25
```

start מאפס את bcd; כל המרה אורכת 16 מחזורי הזזה לאחר start: 65535, 12345, 67890 שגולש ל-2354 ו-20 ערכים אקראיים; 25/25 בדיקות עוברות. bcd מוצג כ-5 nibbles שנקראים ישירות כספרות העשרוניות.



איור 16: tb_binary_2_bcd – זום: המרת 65535.

(1) start=1 מאפס את רגיסטרי הספרות והמונה (2); בכל אחד מ-16 השעונים נכנס ביט אחד של sw, MSB תחילה, עם add-3 לכל ספרה ≥ 5 (3, 4); כשהמונה מגיע ל-16 shift_16 מקפיא את התוצאה (5) 5 3 5 5 6 והבדיקה עוברת (6).



איור 17: tb_binary_2_bcd – זום: המרת 12345.

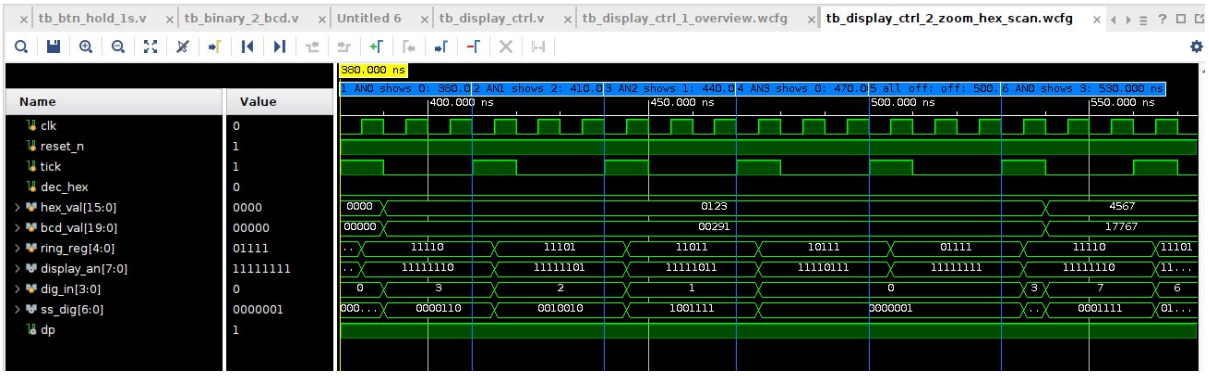
(1) start ואיפוס (2), ההזזה הראשונה (3), וחמשת רגיסטרי הספרות נבנים ל-5 4 3 2 1 עד (4) shift_16; הבדיקה עוברת (5).

4.6. tb_display_ctrl – בדיקת display_ctrl + seven_segment

פלט הטסטבנץ' (tb_display_ctrl_xsim.log):

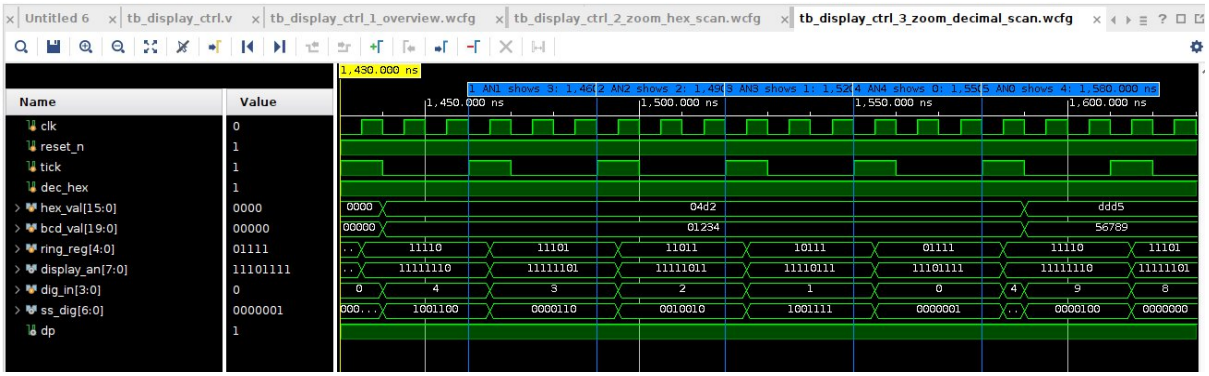
```
Time:    0 | Applied reset
Time:   40 | display_an after reset | actual=11111110 | expected=11111110 | PASS
Time:  240 | display_an stable without tick | actual=11111110 | expected=11111110 | PASS
Time:  270 | display_an after tick | actual=11111101 | expected=11111101 | PASS
  ... (522 more checks) ...
Time: 8520 | TEST PASS | 525/525
```

לאחר reset ring_reg=11110 ו-AN0 דולקת; ה-tick הראשון מתחיל את הסריקה; מצב hex עם 7 ערכים (AN4 לעולם לא דולקת), ואז dec_hex=1 ומצב עשרוני עם 5 ערכים; מעבר ל-hex בזמן ש-AN4 פעילה; 40 ערכים אקראיים לסירוגין בין שני המצבים; 525/525 בדיקות עוברות.



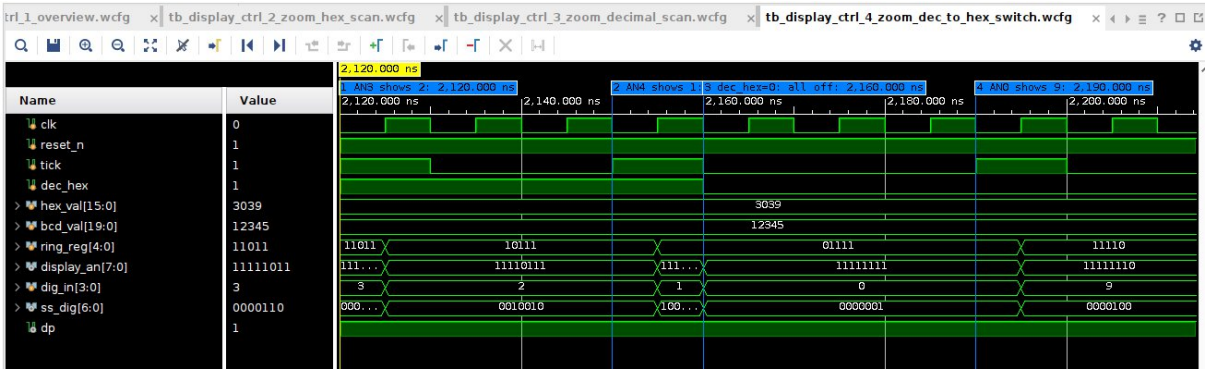
איור 18: tb_display_ctrl – זום: סריקה במצב hex.

סריקת hex של 0123: בכל tick מסתובבים ring_reg ו-display_an (11111110 → 11111101 → 11111011 → 11110111) והסמנים מציינים איזו אנודה דולקת ואיזו ספרה מוצגת עליה; במקום החמישי ring[4]=0 אך dec_hex=0 ולכן display_an=11111111 (הכול כבוי) והסריקה מתחילה מחדש מ-AN0.



איור 19: tb_display_ctrl – זום: סריקה במצב עשרוני.

סריקה עשרונית של AN0–AN3 (bcd 01234): 1234 מציגות 4, 3, 2, 1 ו-(11101111) AN4 מציגה את ספרת עשרות האלפים 0, כמצוין בסמנים.



איור 20: tb_display_ctrl – זום: מעבר ממצב עשרוני ל-hex.

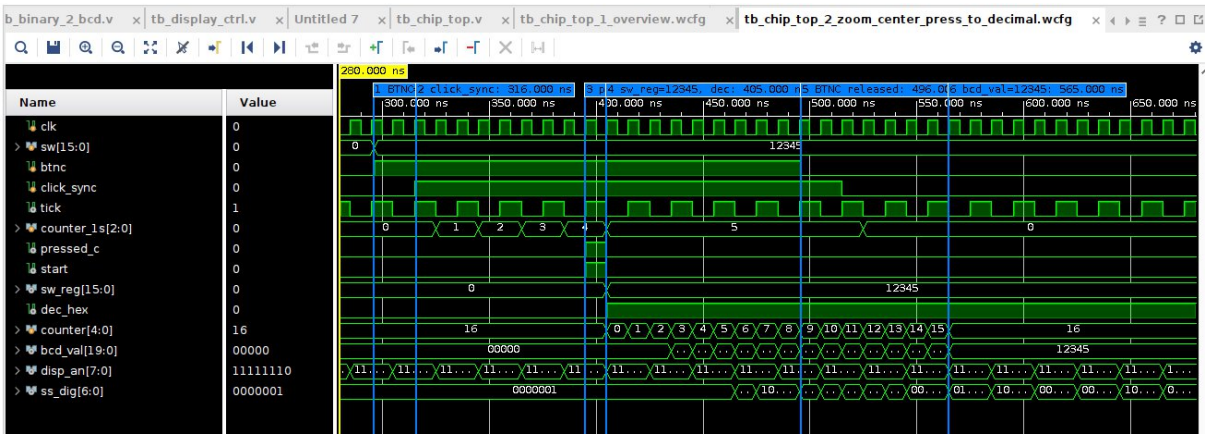
בזמן ש-AN4 היא הספרה הפעילה dec_hex יורד ל-0 ו-display_an הופך ל-11111111 באותו מחזור (הסמן dec_hex=0) – ספרה חמישית ישנה אינה מוצגת לעולם במצב hex; הסריקה ממשיכה מ-AN0.

4.7. tb_chip_top – בדיקת המערכת המלאה

פלט הטסטבנץ' (tb_chip_top_xsim.log):

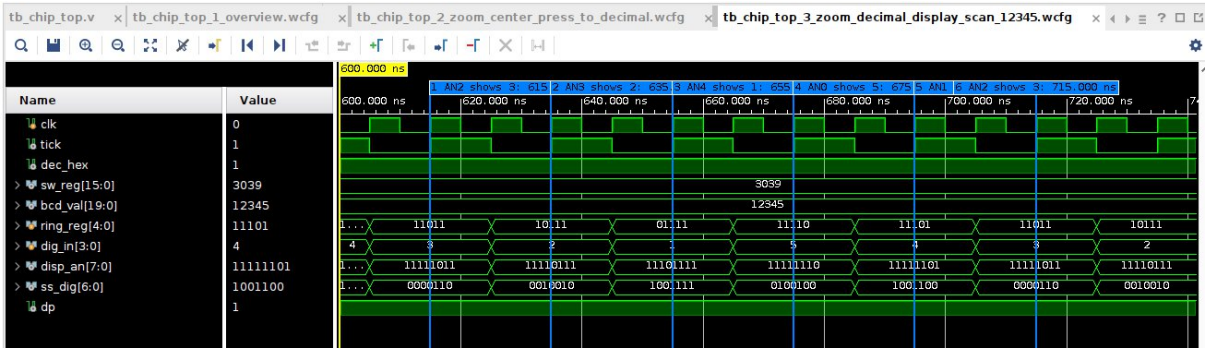
```
Time:   296 | Check display 0 | actual=0 | expected=0 | PASS
Time:   296 | Check display 1 | actual=0 | expected=0 | PASS
Time:   296 | Check display 2 | actual=0 | expected=0 | PASS
Time:   296 | Check display 3 | actual=0 | expected=0 | PASS
  ... (185 more checks) ...
Time: 11446 | TEST PASS | 189/189
```

MAX_TICK=2, MAX_COUNT=5 ("שנייה" = SW=12345 :(100ns ו-BTNC מוחזק, pressed_c דוגם ובוחר עשרוני; BTND מציג 777 ב-hex; לחיצה קצרה מתעלמת; 65535 בעשרוני ו-FFFF ב-hex; 0 בעשרוני; שני הכפתורים יחד – 4321; hex בעשרוני; reset מחזיר ל-10; 0 ערכים אקראיים ו-189/189 בדיקות עוברות. הטסטבנץ' רושם לכל אנודה איזו ספרה הוצגה עליה ומשווה לערך הצפוי.



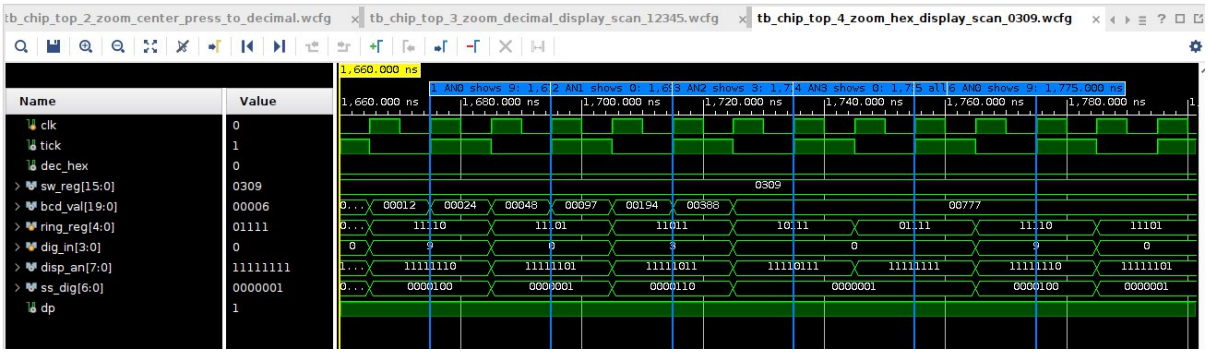
איור 21: tb_chip_top – זום: לחיצה מרכזית ומעבר לעשרוני.

BTNC עולה (1) ומסונכרן (2), חמישה ticks נספרים ו-pressed_c=start פולס למחזור אחד (3); בעלייה הבאה sw_reg=12345, dec_hex=1 והממיר מאופס (4); הכפתור משוחרר 16 ;(5) שעונים לאחר ה-start מתקבל (6) bcd_val=12345 והתצוגה סורקת 5 ספרות.



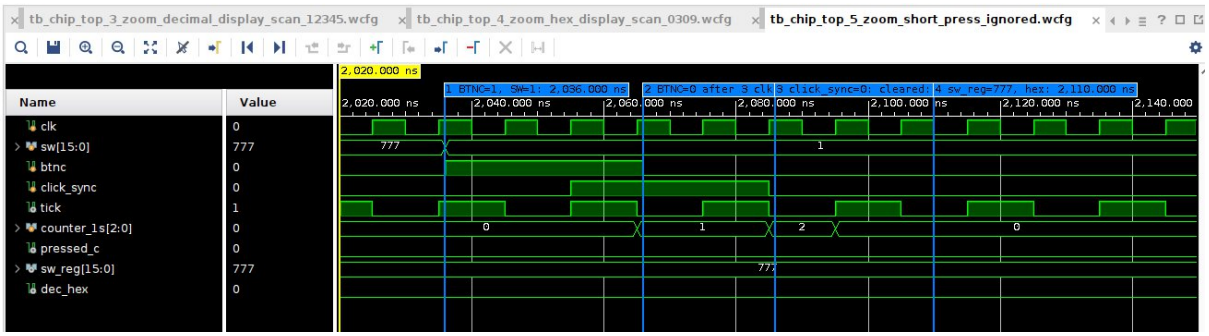
איור 22: tb_chip_top – זום: סריקת התצוגה העשרונית של 12345.

אנודה אחת בכל tick; הסמנים מציינים את AN0–AN4 ואת הספרות 1 2 3 4 5 המוצגות עליהן; DP נשאר כבוי.



איור 23: tb_chip_top – זום: סריקת התצוגה ההקסדצימלית של 0x0309.

סריקת hex של 0x0309 = 777 לאחר לחיצה תחתונה: AN0–AN3 מציגות 0 3 0 9 ובמקום החמישי כל האנודות כבויות (all off).



איור 24: tb_chip_top – זום: לחיצה קצרה מתעלמת.

לחיצת BTNC של 3 מחזורים click_sync: (1, 2) יורד והמונה מתאפס (3) לפני שהגיע ל-5, pressed_c נשאר sw_reg, 0 שומר על 777 והמצב נשאר (4) hex.

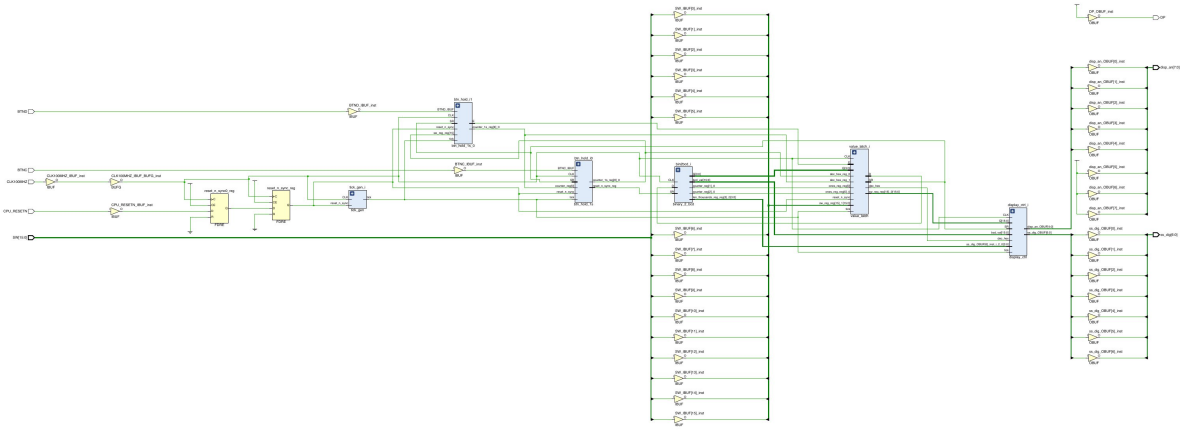
5. סינתזה ומימוש (Synthesis & Implementation)

5.1. סינתזה

הסינתזה (synth_chip_top) הסתיימה עם 0 שגיאות, 0 critical warnings ו-4 אזהרות [[Synth 8-3917 על פורטים המחוברים לקבוע '1' (DP, disp_an[7:5]) – מוצאים שכבויים בכוונה, ולכן הדוח נקי. דוח ה-Methodology מציג 34 אזהרות TIMING-18 (חסר input/output delay) על כל הפינים מלבד השעון; מדובר בכפתורים, מתגים ומוצאי LED ללא יחס תזמון לשעון, ולכן אילוצי input/output delay אינם רלוונטיים עבורם והאזהרה אינפורמטיבית בלבד.

טבלה 13: תוצאות הסינתזה ודוחות ה-Synthesized Design.

Report	Result
Messages (synthesis_run.log)	0 errors, 0 critical warnings, 4 warnings – [Synth 8-3917] port driven by constant 1: DP, disp_an[5], disp_an[6], disp_an[7]
Timing summary (timing_summary_synth.rpt)	sys_clk_pin 10.000 ns (100 MHz): WNS +5.262 ns, WHS +0.144 ns, WPWS +4.500 ns, 0 failing endpoints of 225 – met
DRC (drc_synth.rpt)	0 checks
Methodology (methodology_synth.rpt)	34 x TIMING-18 – missing input/output delay on the asynchronous I/O (informational)
Clock networks / clock interaction	one clock, CLK100MHZ via BUFG, no clock interaction issues
Noise (noise_synth.rpt)	all 36 pins PASS
Power (power_synth.rpt)	0.110 W total on-chip (estimate)



איור 25: סכמת ה-post-synthesis של chip_top.

5.2. מימוש (Implementation)

ה-Implementation (opt_design, place_design, route_design) הסתיים עם 0 שגיאות, 0 critical warnings ו-0 אזהרות. כל הנתיבים עומדים באילוצי התזמון ב-100MHz עם מרווח של 5.326ns, כל 184 הרשתות נותבו ללא שגיאות ודוח ה-DRC נקי. 34 אזהרות ה-TIMING-18 בדוח ה-Methodology זהות לאלו שלאחר הסינתזה. סכמות ה-post-implementation זהות לסכמות ה-post-synthesis (מלבד 4 עותקי ה-lopt_replica של ring_reg), ולכן מובאת רק סכמת ה-chip_top שלאחר הסינתזה (איור 25).

טבלה 14: תוצאות המימוש ודוחות ה-Implemented Design.

Report	Result
Messages (implementation_run.log)	0 errors, 0 critical warnings, 0 warnings
Timing summary (timing_summary_routed.rpt)	WNS +5.326 ns, TNS 0 (0 of 237 endpoints failing); WHS +0.163 ns, THS 0; WPWS +4.500 ns – all constraints met
Critical path	tick_gen_i/counter_reg[3] → value_latch_i/sw_reg_reg[11]/CE: 4.397 ns, 4 logic levels (logic 0.952 ns, routing 3.445 ns)
Route status (route_status.rpt)	288 logical nets, 184 routable nets, 184 fully routed, 0 nets with routing errors
DRC (drc_routed.rpt)	0 checks
Methodology (methodology_drc_routed.rpt)	34 x TIMING-18 – same I/O-delay note as after synthesis
Power (power_routed.rpt)	0.109 W total: 0.097 W static + 0.011 W dynamic; junction 25.5 °C
I/O (io_placed.rpt)	all 36 ports placed on the XDC pins, LVCMOS33 (SW[8], SW[9] LVCMOS18), no unconstrained ports

6. ניצול משאבים

הערכים נלקחו מקובצי ה-.rpt (utilization_synth.rpt, utilization_placed.rpt) ודוחות ה-out-of-context של כל מודול בתיקייה Resource_Usage_5), הסופרים slice LUTs לאחר LUT combining. רוב הלוגיקה נכתבה בקוד התנהגותי (מונים, בוררים ו-case), והמיפוי ל-LUT, ל-CARRY4 ול-MUXF7/MUXF8 נקבע על ידי הסינתזה.

טבלה 15: ניצול משאבים של העיצוב המלא לאחר סינתזה ולאחר Place & Route.

Site type	Synthesis	Placed	Available	Util % (placed)
Slice LUTs (all as logic)	83	80	63,400	0.13
Slice Registers (all flip-flops)	91	95	126,800	0.07
Register as Latch	0	0	126,800	0.00
Slices (27 SLICEL + 8 SLICEM)	–	35	15,850	0.22
F7 Muxes	1	1	31,700	<0.01
Block RAM Tile / DSPs	0 / 0	0 / 0	135 / 240	0.00
Bonded IOB (20 inputs + 16 outputs)	36	36	210	17.14
BUFGCTRL	1	1	32	3.13

Primitives (synthesis / placed): FDRE 87 / 88, FDSE 4 / 7, LUT6 37, LUT4 37, LUT5 16, LUT2 11, LUT3 6, LUT1 3 (110 LUT primitives = 83 / 80 slice LUTs after LUT combining; after placement 50 LUTs use O6 only and 30 use O5 and O6), CARRY4 4, MUXF7 1, IBUF 20, OBUF 16, BUFG 1. Unique control sets (placed): 9.

טבלה 16: ניצול משאבים לפי מודול (out-of-context synthesis) ובעיצוב המלא.

Module	Registers in the RTL	FF	Slice LUTs	Other
chip_top (top level)	reset_n_sync0, reset_n_sync	2	0	–
tick_gen	counter[15:0]	16	7	4 CARRY4
btn_hold_1s (x2)	click_sync0, click_sync, counter_1s[10:0]	13 each (26)	15 each (30)	–
value_latch	sw_reg[15:0], dec_hex	17	3	–
binary_2_bcd	5 x 4-bit digit registers, counter[4:0]	25	17	2 MUXF7, 1 MUXF8
display_ctrl (incl. seven_segment)	ring_reg[4:0]	5	37	–
seven_segment alone	–	0	7	–
Sum of modules		91	94	
Full design: synth / placed		91 / 95	83 / 80	1 BUFG

העיצוב המלא (80 LUT) קטן מסכום המודולים (94) בזכות propagation של קבועים (DP, AN7–AN5) ושיתוף לוגיקת ה-tick וה-reset בין שני מופעי btn_hold_1s; 95 הדלגלגים לאחר ה-Implementation כוללים 4 עותקים (lopt_replica) של ring_reg[3:0] שיצר opt_design לפיצול ה-fanout. הניצול הכולל זניח ביחס לרכיב, ולא הופיעו BRAM, DSP, latches או מאגרי שעון נוספים – כמתאים לעיצוב קטן בתחום שעון יחיד.

7. בדיקה על הלוח

7.1. יצירת ה-bitstream וטעינתו ללוח

יצירת ה-bitstream (chip_top.bit, Generate Bitstream) על impl_chip_top הסתיימה עם 0 שגיאות, 0 critical warnings ו-0 אזהרות. הקובץ נטען ללוח ה-Nexys A7-100T דרך JTAG מתוך ה-Hardware Manager, ונורית ה-DONE נדלקה עם סיום הטעינה (איור 26).

טבלה 17: סיכום יצירת ה-bitstream.

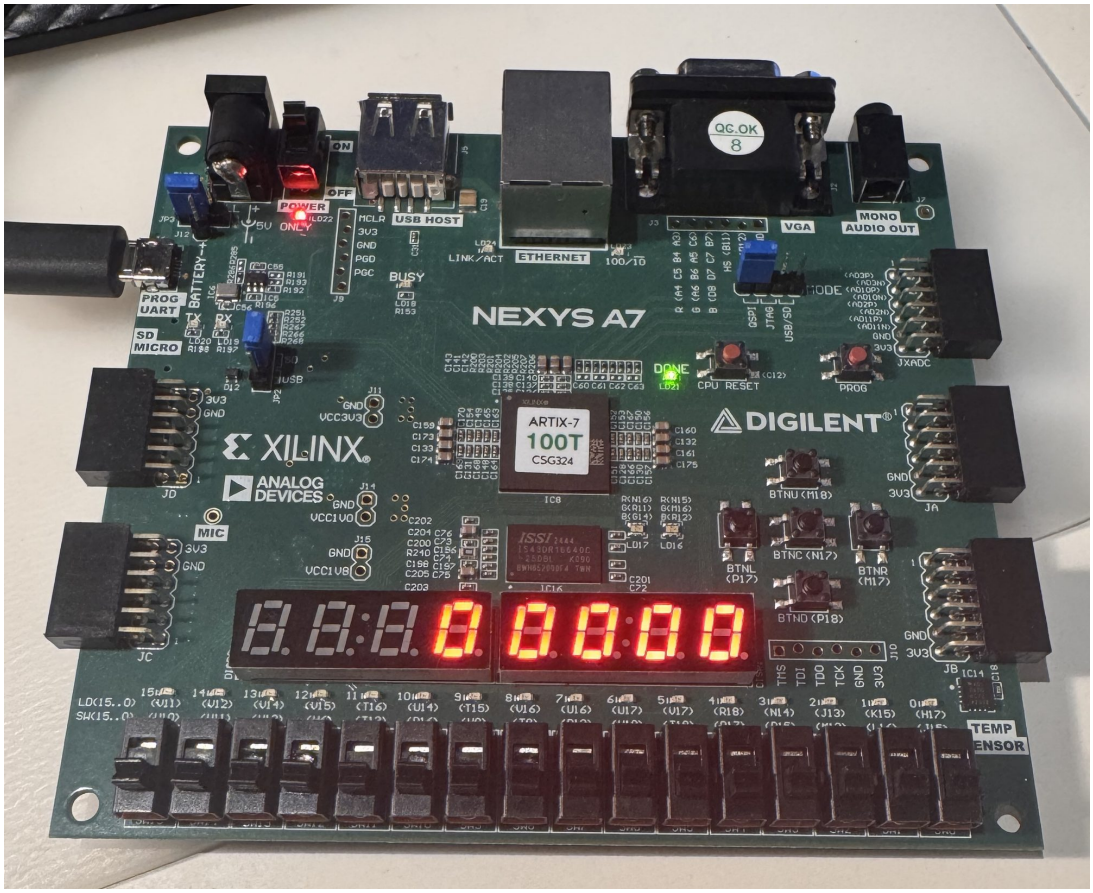
Item	Result
Bitstream	chip_top.bit – 0 errors, 0 critical warnings, 0 warnings (6_Bitstream/bitstream_summary.txt)
Device	xc7a100tcsg324-1 (Digilent Nexys A7-100T)
Clock constraint	create_clock sys_clk_pin, period 10.000 ns on CLK100MHZ
Programming	JTAG via the on-board USB port (Hardware Manager, Program Device); DONE LED on

7.2. בדיקה פונקציונלית על הלוח

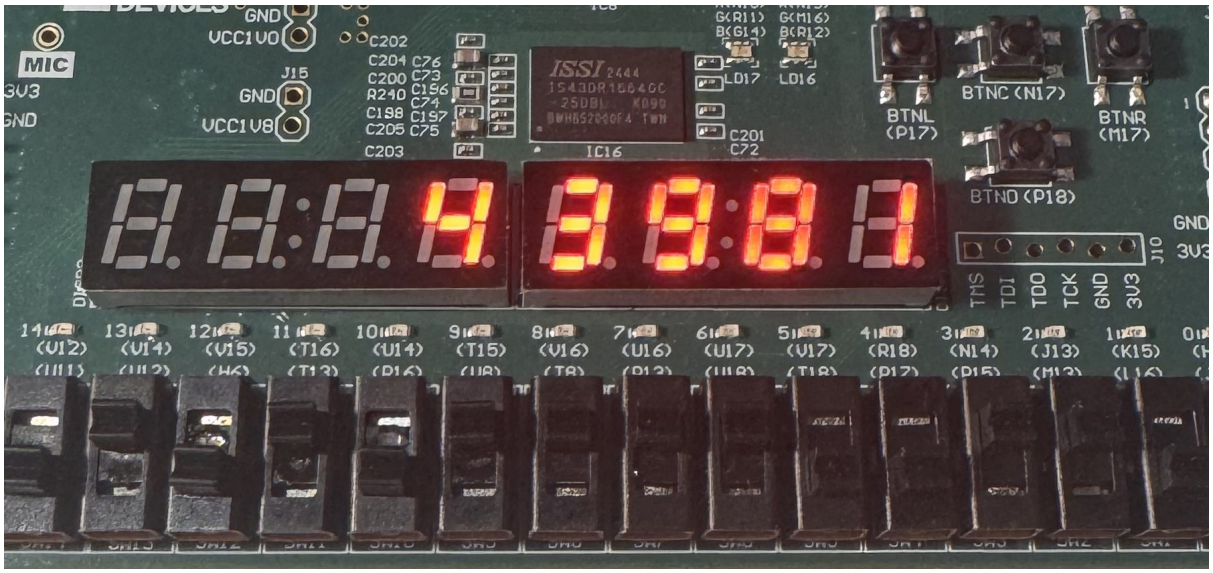
הבדיקה על הלוח חזרה על תרחישי tb_chip_top: לכל ערך מתגים הוחזק BTNC למשך שנייה (ייצוג עשרוני) ולאחר מכן BTND (ייצוג הקסדצימלי), והספרות שהוצגו הושוו לערך הצפוי. בכל המקרים התצוגה יציבה וללא הבהוב, הספרה החמישית (AN4) דולקת רק בייצוג העשרוני, לחיצה קצרה אינה משנה את הערך, וכפתור ה-CPU RESET מחזיר את התצוגה ל-0000. טבלה 18 מרכזת את המקרים שנבדקו, ואיורים 32–26 מציגים את התצוגה בכל מקרה.

טבלה 18: מקרי הבדיקה על הלוח.

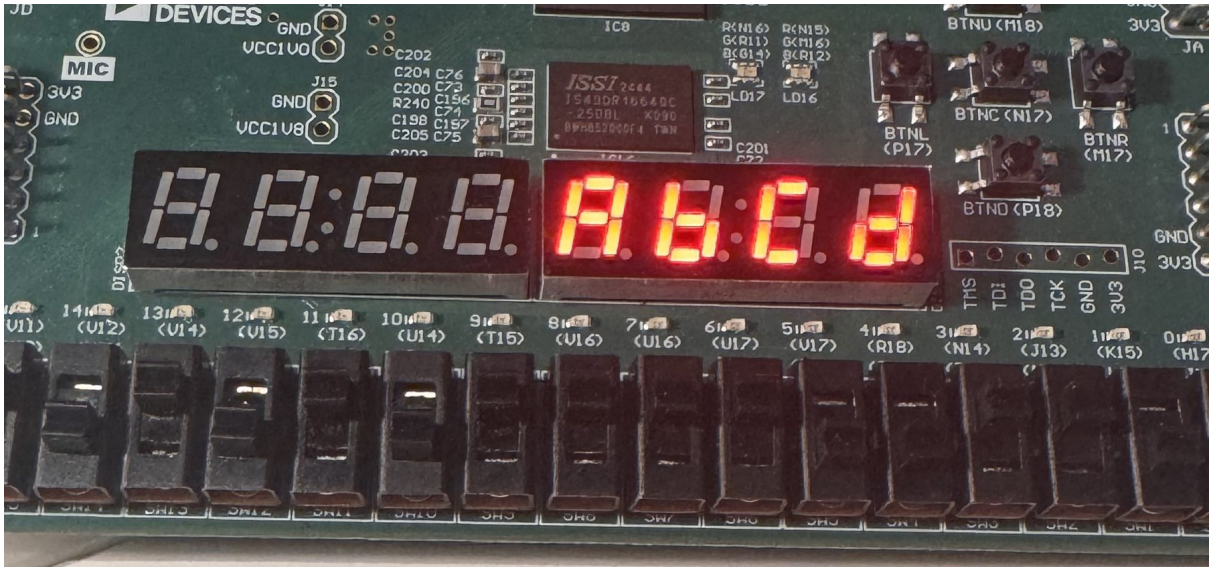
Case	SW[15:0]	Button held 1 s	Mode	Display	Result	Figure
1	0x0000	BTNC	Decimal	00000	Pass	26
2	0xABCD	BTNC	Decimal	43981	Pass	27
3	0xABCD	BTND	Hex	AbCd	Pass	28
4	0xFFFF	BTNC	Decimal	65535	Pass	29
5	0xFFFF	BTND	Hex	FFFF	Pass	30
6	0x1234	BTNC	Decimal	04660	Pass	31
7	0x1234	BTND	Hex	1234	Pass	32



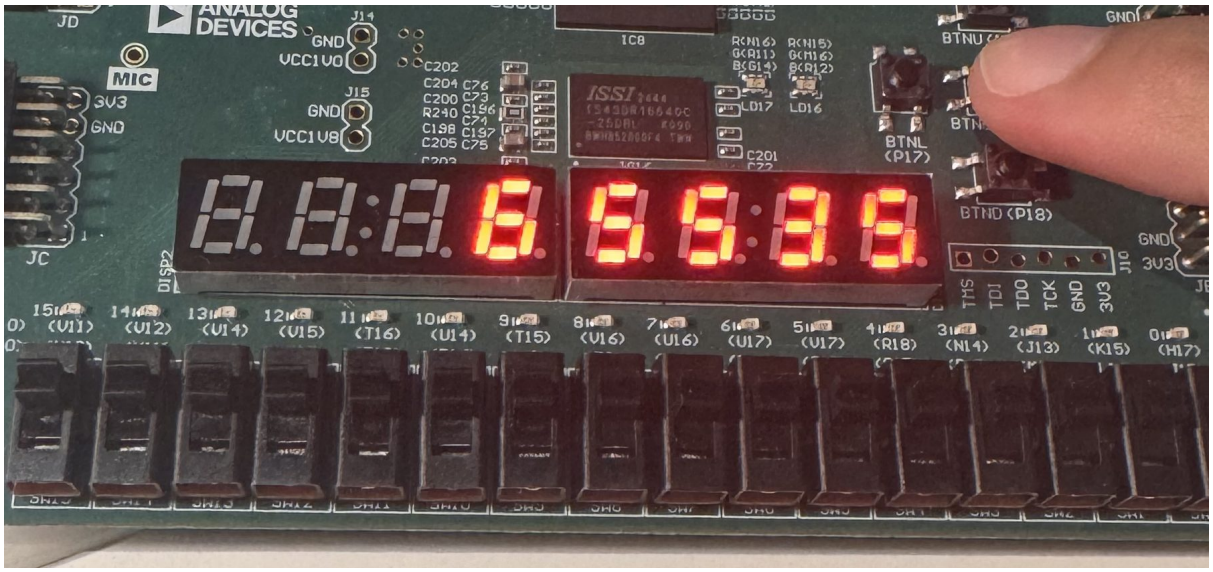
איור 26: הלוח לאחר טעינת ה-bitstream: נורית ה-DONE דולקת והתצוגה מראה 00000 – הערך 0 בייצוג עשרוני על 5 ספרות.



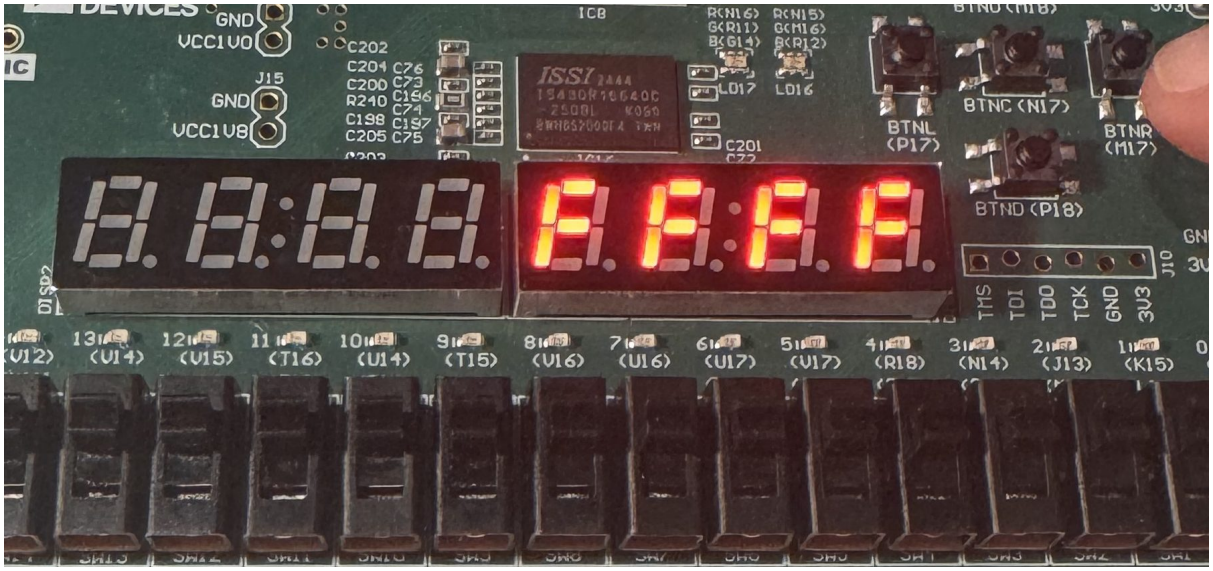
איור 27: SW = 0xABCD לאחר החזקת BTNC: התצוגה מראה 43981 בייצוג עשרוני.



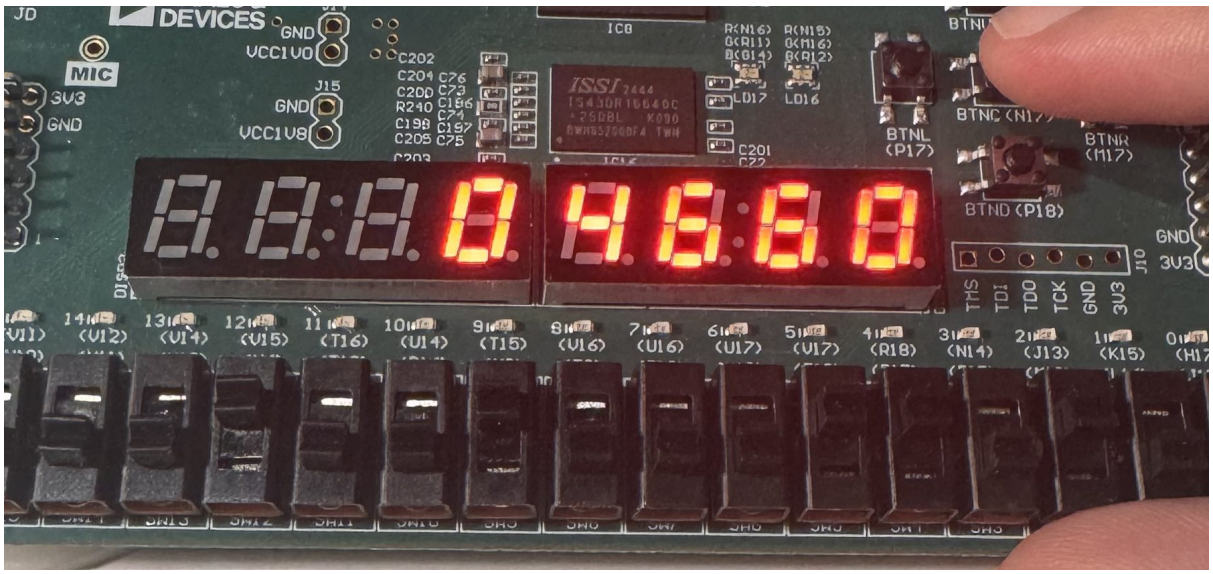
איור 28: SW = 0xABCD לאחר החזקת BTND: התצוגה מראה AbCd בייצוג הקסדצימלי – 4 ספרות, AN4 כבויה.



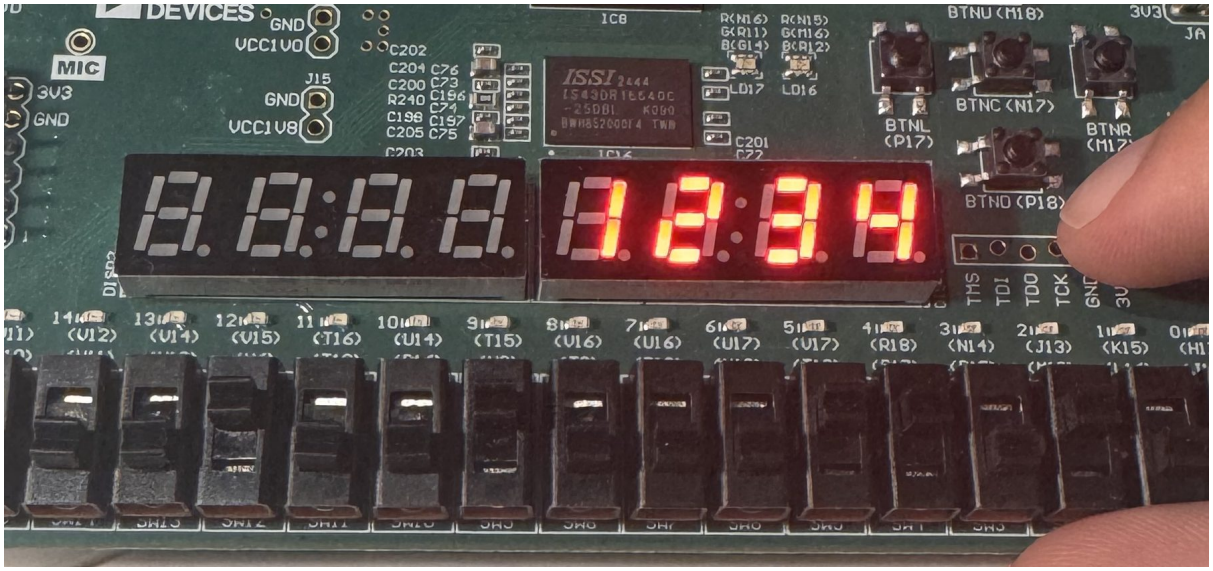
איור 29: SW = 0xFFFF לאחר החזקת BTNC: 65535 – הערך המקסימלי, 5 ספרות.



איור 30: SW = 0xFFFF לאחר החזקת BTND: FFFF.



איור 31: SW = 0x1234 לאחר החזקת BTNC: 04660 – הספרה החמישית מציגה את האפס המוביל.



איור 32: SW = 0x1234 לאחר החזקת BTND: 1234.

8. סיכום

בקר תצוגת שבעת המקטעים מומש בשבעה מודולים סינכרוניים קטנים על שעון יחיד של 100MHz, עם דגימת ערך המתגים לאחר החזקה של שנייה, ייצוג עשרוני או הקסדצימלי וקצב רענון של 400Hz ללא הבהוב. כל מודול והמערכת כולה נבדקו בטסטבנצ'ים הבודקים את עצמם (798 בדיקות, 0 כישלונות), ובצורות הגל נראים כל הדרישות: הדגימה, זמן ההחזקה, פולס יחיד ללחיצה, המרת ה-BCD, סריקת 5 הספרות והספרה החמישית רק במצב עשרוני. ה-Lint, בדיקות ה-Elaborated Design, הסינתזה, המימוש ויצירת ה-bitstream נקיים; התזמון נסגר עם מרווח של 5.3ns, והניצול (95 ,80 slice LUTs דלגלגים, 35 slices) זניח ביחס לרכיב. על הלוח הוצגו הערכים 0xFFFF, 0xABCD, 0x0000 ו-0x1234 בשני הייצוגים כנדרש.

נספח א': קוד המקור (RTL ואילוצים)

קוד המקור של כל המודולים (תיקיית src) וקטע האילוצים הפעיל מקובץ ה-XDC של הלוח:

chip_top.v

```
`timescale 1ns/1ps
module chip_top #(
   parameter MAX_TICK = 50000,
   parameter MAX_COUNT = 2000
 ) (
   input CLK100MHZ, CPU_RESETN,
   input [15:0] SW,
   input BTNC, BTND,
   output DP,
   output [6:0] ss_dig,
   output [7:0] disp_an
);
   wire tick;
   wire pressed_c, pressed_d;

   wire start;
   wire [19:0] bcd_val;

   wire dec_hex;
   wire [15:0] sw_reg;

   reg reset_n_sync = 1'b0, reset_n_sync0 = 1'b0;
   always @(posedge CLK100MHZ) begin
      reset_n_sync0 <= CPU_RESETN;
      reset_n_sync <= reset_n_sync0;
   end

   tick_gen #(.MAX_TICK(MAX_TICK)) tick_gen_i (CLK100MHZ, reset_n_sync, tick);

   // Center buton
   btn_hold_1s #(.MAX_COUNT(MAX_COUNT)) btn_hold_i0 (CLK100MHZ, reset_n_sync, BTNC, tick,
pressed_c);
   // Down buton
   btn_hold_1s #(.MAX_COUNT(MAX_COUNT)) btn_hold_i1 (CLK100MHZ, reset_n_sync, BTND, tick,
pressed_d);

   value_latch value_latch_i (CLK100MHZ, reset_n_sync, pressed_c, pressed_d, SW, dec_hex, sw_reg);

   assign start = pressed_c | pressed_d;

   binary_2_bcd bin2bcd_i (CLK100MHZ, reset_n_sync, sw_reg, start, bcd_val);

   display_ctrl display_ctrl_i (CLK100MHZ, reset_n_sync, tick, dec_hex, sw_reg, bcd_val, disp_an, ss_dig, DP);

endmodule
```

tick_gen.v

```
`timescale 1ns/1ps
```

```
module tick_gen #(parameter MAX_TICK = 50000) (
   input clk, reset_n,
   output tick
);
   localparam TICK_BITS = $clog2(MAX_TICK);
   localparam [TICK_BITS-1:0] MAX_CNT = MAX_TICK -1;
   localparam [TICK_BITS-1:0] ONE = 1;

   reg  [TICK_BITS-1:0] counter;

   assign tick = counter == MAX_CNT;

   always @(posedge clk) begin
      if (!reset_n | tick) counter <=  {TICK_BITS{1'b0}};
      else counter <= counter + ONE;
   end


endmodule
```

btn_hold_1s.v

```
`timescale 1ns/1ps
module btn_hold_1s #(parameter MAX_COUNT = 2000) (
   input clk, reset_n,
   input click,
   input tick,
   output pressed_1s
);

   // Need a synchronizer for click because is is async coming into a synch system
   reg click_sync, click_sync0;
   always @(posedge clk) begin
      if (!reset_n) {click_sync, click_sync0} <= 2'b00;
      else begin
         click_sync0 <= click;
         click_sync <= click_sync0;
      end
   end

   // Need to gen a 1s tick counter from the 400 tick gen
   // 2kHz * 1/1Hz = 2000 -> Need to count 2000 ticks of tick. need log2(2000+1) = 10.96 -> reg [10:0] (11 bits)
   // we need 2001 because their is no wrap around so we dont count it, and need to compinsate
   localparam MAX_COUNT_BITS = $clog2(MAX_COUNT + 1);
   localparam [MAX_COUNT_BITS-1:0] MAX_CNT_M1 = MAX_COUNT - 1;
   localparam [MAX_COUNT_BITS-1:0] MAX_CNT = MAX_COUNT;
   localparam [MAX_COUNT_BITS-1:0] ONE = 1;

   reg [MAX_COUNT_BITS-1:0] counter_1s;

   always @(posedge clk) begin
      if (!reset_n | !click_sync) counter_1s <= {MAX_COUNT_BITS{1'b0}};
      else if (tick) begin
         if (counter_1s == MAX_CNT) counter_1s <= counter_1s;
         else counter_1s <= counter_1s + ONE;
```

```
    end
  end

  assign pressed_1s = click_sync & tick & (counter_1s == MAX_CNT_M1);


endmodule
```

value_latch.v

```
`timescale 1ns/1ps
module value_latch (
  input clk, reset_n,
  input pressed_c, pressed_d,
  input [15:0] sw,
  output reg dec_hex,    // When 1 decimal is pressed, when 0 hex is pressed, reseted to hex view
  output reg [15:0] sw_reg
);
  always @(posedge clk) begin
    if (!reset_n | pressed_d) dec_hex <= 1'b0;
    else if (pressed_c) dec_hex <= 1'b1;
  end

  always @(posedge clk) begin
    if (!reset_n) sw_reg <= 16'b0;
    else if (pressed_c | pressed_d) sw_reg <= sw;
  end

endmodule
```

binary_2_bcd.v

```
`timescale 1ns/1ps
module binary_2_bcd (
  input clk, reset_n,
  input [15:0] sw,
  input start,
  output [19:0] bcd
);

  // Double Dabble algorithim.
  // If a part (ones/tens/hundreds..) >= 5 then we need to add 3 to that number
  // No need to check ten_thousands because max digit is 6, so before last shift is was 3 (0011) -> (0110) so no need to check add 3 (3 smaller then 5 will never be more)
  // one big shift regesiter between the added parts
  // counter to make sure we do 16 shifts and thats the end.
  // can also add edge case where sw == 16'b0 and reduce calculation time. but wont see a difference on screen because calculation time is 160 ns.
  wire [3:0] ones, tens, hundreds, thousands;
  reg [3:0] ones_reg, tens_reg, hundreds_reg, thousands_reg, ten_thousands_reg;
  reg [4:0] counter;

  assign ones = (ones_reg >= 4'd5) ? ones_reg + 4'd3 : ones_reg;
```

```
    assign tens = (tens_reg >= 4'd5) ? tens_reg + 4'd3 : tens_reg;
    assign hundreds = (hundreds_reg >= 4'd5) ? hundreds_reg + 4'd3 : hundreds_reg;
    assign thousands = (thousands_reg >= 4'd5) ? thousands_reg + 4'd3 : thousands_reg;

    wire shift_16 = counter == 5'd16;

    always @(posedge clk) begin
        if (!reset_n | start) counter <= 5'b0;
        else begin
            if (shift_16) counter <= counter;
            else counter <= counter + 5'd1;
        end
    end

    always @(posedge clk) begin
        if (!reset_n | start) begin
            {ten_thousands_reg, thousands_reg, hundreds_reg, tens_reg, ones_reg} <= 20'b0;
        end
        else if (!shift_16) begin
            ones_reg <= {ones[2:0], sw[15-counter]};
            tens_reg <= {tens[2:0], ones[3]};
            hundreds_reg <= {hundreds[2:0], tens[3]};
            thousands_reg <= {thousands[2:0], hundreds[3]};
            ten_thousands_reg <= {ten_thousands_reg[2:0], thousands[3]};
        end
    end

    assign bcd = {ten_thousands_reg, thousands_reg, hundreds_reg, tens_reg, ones_reg};
endmodule
```

display_ctrl.v

```
`timescale 1ns/1ps
module display_ctrl (
    input clk, reset_n,
    input tick, // 2kHz tick
    input dec_hex,
    input [15:0] hex_val,
    input [19:0] bcd_val,
    output [7:0] display_an,
    output [6:0] ss_dig,
    output dp
);

    reg [4:0] ring_reg;

    always @(posedge clk) begin
        if (!reset_n) ring_reg <= 5'b11110;
        else if (tick) ring_reg <= {ring_reg[3:0], ring_reg[4]};
    end

    assign display_an = {3'b111, !dec_hex | ring_reg[4], ring_reg[3:0]}; // needed to be zero when dec_hex is 1 and ring[4] is 0 -> dec_hex & !ring[4] = 1 -> !(dec_hex & ring[4]) = !dec_hex | ring[4] = 0

    wire [19:0] val_to_show = (dec_hex) ? bcd_val : {4'b0, hex_val};
```

```
    reg [3:0] dig_in;
    always @(*) begin
        case (ring_reg)
            5'b11110: dig_in = val_to_show[3:0];
            5'b11101: dig_in = val_to_show[7:4];
            5'b11011: dig_in = val_to_show[11:8];
            5'b10111: dig_in = val_to_show[15:12];
            5'b01111: dig_in = val_to_show[19:16];
            default: dig_in = val_to_show[3:0];
        endcase
    end

    seven_segment ss_i (dig_in, ss_dig);

    assign dp = 1'b1;

endmodule
```

seven_segment.v

```
`timescale 1ns/1ps
module seven_segment (
    input [3:0] dig_in,
    output reg [6:0] ss_dig
);

    always @(*) begin
        case (dig_in)
            4'h0: ss_dig = 7'b0000001;
            4'h1: ss_dig = 7'b1001111;
            4'h2: ss_dig = 7'b0010010;
            4'h3: ss_dig = 7'b0000110;
            4'h4: ss_dig = 7'b1001100;
            4'h5: ss_dig = 7'b0100100;
            4'h6: ss_dig = 7'b0100000;
            4'h7: ss_dig = 7'b0001111;
            4'h8: ss_dig = 7'b0000000;
            4'h9: ss_dig = 7'b0000100;
            4'hA: ss_dig = 7'b0001000;
            4'hB: ss_dig = 7'b1100000;
            4'hC: ss_dig = 7'b0110001;
            4'hD: ss_dig = 7'b1000010;
            4'hE: ss_dig = 7'b0110000;
            4'hF: ss_dig = 7'b0111000;
            default: ss_dig = 7'b1111111;
        endcase
    end

endmodule
```

NexysA7-100T_Master.xdc (האילוצים הפעילים בלבד)

```
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property -dict { PACKAGE_PIN E3    IOSTANDARD LVCMOS33 } [get_ports { CLK100MHZ }];
#IO_L12P_T1_MRCC_35 Sch=clk100mhz
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports {CLK100MHZ}];
set_property -dict { PACKAGE_PIN J15   IOSTANDARD LVCMOS33 } [get_ports { SW[0] }];
#IO_L24N_T3_RS0_15 Sch=sw[0]
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { SW[1] }];
#IO_L3N_T0_DQS_EMCCLK_14 Sch=sw[1]
set_property -dict { PACKAGE_PIN M13   IOSTANDARD LVCMOS33 } [get_ports { SW[2] }];
#IO_L6N_T0_D08_VREF_14 Sch=sw[2]
set_property -dict { PACKAGE_PIN R15   IOSTANDARD LVCMOS33 } [get_ports { SW[3] }];
#IO_L13N_T2_MRCC_14 Sch=sw[3]
set_property -dict { PACKAGE_PIN R17   IOSTANDARD LVCMOS33 } [get_ports { SW[4] }];
#IO_L12N_T1_MRCC_14 Sch=sw[4]
set_property -dict { PACKAGE_PIN T18   IOSTANDARD LVCMOS33 } [get_ports { SW[5] }]; #IO_L7N_T1_D10_14
Sch=sw[5]
set_property -dict { PACKAGE_PIN U18   IOSTANDARD LVCMOS33 } [get_ports { SW[6] }];
#IO_L17N_T2_A13_D29_14 Sch=sw[6]
set_property -dict { PACKAGE_PIN R13   IOSTANDARD LVCMOS33 } [get_ports { SW[7] }];
#IO_L5N_T0_D07_14 Sch=sw[7]
set_property -dict { PACKAGE_PIN T8    IOSTANDARD LVCMOS18 } [get_ports { SW[8] }]; #IO_L24N_T3_34
Sch=sw[8]
set_property -dict { PACKAGE_PIN U8    IOSTANDARD LVCMOS18 } [get_ports { SW[9] }]; #IO_25_34
Sch=sw[9]
set_property -dict { PACKAGE_PIN R16   IOSTANDARD LVCMOS33 } [get_ports { SW[10] }];
#IO_L15P_T2_DQS_RDWR_B_14 Sch=sw[10]
set_property -dict { PACKAGE_PIN T13   IOSTANDARD LVCMOS33 } [get_ports { SW[11] }];
#IO_L23P_T3_A03_D19_14 Sch=sw[11]
set_property -dict { PACKAGE_PIN H6    IOSTANDARD LVCMOS33 } [get_ports { SW[12] }]; #IO_L24P_T3_35
Sch=sw[12]
set_property -dict { PACKAGE_PIN U12   IOSTANDARD LVCMOS33 } [get_ports { SW[13] }];
#IO_L20P_T3_A08_D24_14 Sch=sw[13]
set_property -dict { PACKAGE_PIN U11   IOSTANDARD LVCMOS33 } [get_ports { SW[14] }];
#IO_L19N_T3_A09_D25_VREF_14 Sch=sw[14]
set_property -dict { PACKAGE_PIN V10   IOSTANDARD LVCMOS33 } [get_ports { SW[15] }];
#IO_L21P_T3_DQS_14 Sch=sw[15]
set_property -dict { PACKAGE_PIN T10   IOSTANDARD LVCMOS33 } [get_ports { ss_dig[6] }];
#IO_L24N_T3_A00_D16_14 Sch=ca
set_property -dict { PACKAGE_PIN R10   IOSTANDARD LVCMOS33 } [get_ports { ss_dig[5] }]; #IO_25_14
Sch=cb
set_property -dict { PACKAGE_PIN K16   IOSTANDARD LVCMOS33 } [get_ports { ss_dig[4] }]; #IO_25_15
Sch=cc
set_property -dict { PACKAGE_PIN K13   IOSTANDARD LVCMOS33 } [get_ports { ss_dig[3] }];
#IO_L17P_T2_A26_15 Sch=cd
set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { ss_dig[2] }];
#IO_L13P_T2_MRCC_14 Sch=ce
set_property -dict { PACKAGE_PIN T11   IOSTANDARD LVCMOS33 } [get_ports { ss_dig[1] }];
#IO_L19P_T3_A10_D26_14 Sch=cf
set_property -dict { PACKAGE_PIN L18   IOSTANDARD LVCMOS33 } [get_ports { ss_dig[0] }];
#IO_L4P_T0_D04_14 Sch=cg
set_property -dict { PACKAGE_PIN H15   IOSTANDARD LVCMOS33 } [get_ports { DP }];
#IO_L19N_T3_A21_VREF_15 Sch=dp
set_property -dict { PACKAGE_PIN J17   IOSTANDARD LVCMOS33 } [get_ports { disp_an[0] }];
#IO_L23P_T3_FOE_B_15 Sch=an[0]
```

```
set_property -dict { PACKAGE_PIN J18   IOSTANDARD LVCMOS33 } [get_ports { disp_an[1] }];
#IO_L23N_T3_FWE_B_15 Sch=an[1]
set_property -dict { PACKAGE_PIN T9    IOSTANDARD LVCMOS33 } [get_ports { disp_an[2] }];
#IO_L24P_T3_A01_D17_14 Sch=an[2]
set_property -dict { PACKAGE_PIN J14   IOSTANDARD LVCMOS33 } [get_ports { disp_an[3] }];
#IO_L19P_T3_A22_15 Sch=an[3]
set_property -dict { PACKAGE_PIN P14   IOSTANDARD LVCMOS33 } [get_ports { disp_an[4] }];
#IO_L8N_T1_D12_14 Sch=an[4]
set_property -dict { PACKAGE_PIN T14   IOSTANDARD LVCMOS33 } [get_ports { disp_an[5] }];
#IO_L14P_T2_SRCC_14 Sch=an[5]
set_property -dict { PACKAGE_PIN K2    IOSTANDARD LVCMOS33 } [get_ports { disp_an[6] }]; #IO_L23P_T3_35
Sch=an[6]
set_property -dict { PACKAGE_PIN U13   IOSTANDARD LVCMOS33 } [get_ports { disp_an[7] }];
#IO_L23N_T3_A02_D18_14 Sch=an[7]
set_property -dict { PACKAGE_PIN C12   IOSTANDARD LVCMOS33 } [get_ports { CPU_RESETN }];
#IO_L3P_T0_DQS_AD1P_15 Sch=cpu_resetn
set_property -dict { PACKAGE_PIN N17   IOSTANDARD LVCMOS33 } [get_ports { BTNC }];
#IO_L9P_T1_DQS_14 Sch=btnc
set_property -dict { PACKAGE_PIN P18   IOSTANDARD LVCMOS33 } [get_ports { BTND }];
#IO_L9N_T1_DQS_D13_14 Sch=btnd
```

נספח ב': קוד הטסטבנצ'ים

הטסטבנצ'ים של כל המודולים ושל המערכת המלאה (תיקיית sim), לפי הסדר של טבלה 10:

tb_ss.v

```
`timescale 1ns/1ps
module tb_ss ();

   reg [3:0] dig_in;
   wire [6:0] ss_dig;

   reg [4:0] pass = 0, fail = 0;
   reg [4:0] expected_dig_in;

   seven_segment ss_i (dig_in, ss_dig);
   integer i;

   initial begin
      for (i = 0; i < 16; i = i + 1) begin
         dig_in = i[3:0]; #10;
         expected_dig_in = ss2dig(ss_dig);
         $display("Time:%5d | digit:%1h | decoded:%1h | ss:%b | %s", $time, i, expected_dig_in, ss_dig,
(expected_dig_in==i[4:0]) ? "PASS" : "FAIL");
         if (expected_dig_in == i[4:0]) begin
            pass = pass + 1;
         end else begin
            fail = fail + 1;
         end
      end

      #10;
      if (fail == 0) $display("Time: %0d | TEST PASS %0d/%0d", $time, pass, pass + fail);
      else $display("Time: %0d | TEST FAIL %0d/%0d", $time, fail, pass + fail);
      $finish();
   end

   function [4:0] ss2dig (input [6:0] ss_dig);
      begin
         case(ss_dig)
            7'b0000001: ss2dig = 5'h0;
            7'b1001111: ss2dig = 5'h1;
            7'b0010010: ss2dig = 5'h2;
            7'b0000110: ss2dig = 5'h3;
            7'b1001100: ss2dig = 5'h4;
            7'b0100100: ss2dig = 5'h5;
            7'b0100000: ss2dig = 5'h6;
            7'b0001111: ss2dig = 5'h7;
            7'b0000000: ss2dig = 5'h8;
            7'b0000100: ss2dig = 5'h9;
            7'b0001000: ss2dig = 5'ha;
            7'b1100000: ss2dig = 5'hb;
            7'b0110001: ss2dig = 5'hc;
            7'b1000010: ss2dig = 5'hd;
            7'b0110000: ss2dig = 5'he;
            7'b0111000: ss2dig = 5'hf;
```

```
            default ss2dig = 5'b10000;
        endcase
    end
  endfunction

endmodule
```

tb_tick_gen.v

```
`timescale 1ns/1ps
module tb_tick_gen (
);
  localparam MAX_TICK = 5;
  reg clk = 1'b0;
  always #5 clk = ~clk;

  reg reset_n;
  wire tick;
  reg exp_tick;

  integer pass = 0, fail = 0;

  task reset ();
    begin
      @(negedge clk); reset_n = 1'b0; exp_tick = 1'b0;
      @(negedge clk); reset_n = 1'b1;
    end
  endtask

  tick_gen #(.MAX_TICK(MAX_TICK)) tick_gen_i (clk, reset_n, tick);

  integer cycles = 0;
  reg first_seen = 1'b0;
  always @(negedge clk) begin
    if (reset_n) begin
      if (tick) begin
        if (first_seen) begin
          $display("Time: %5d | Check tick interval correct | actual=%d | expected=%d | %s", $time,
cycles, MAX_TICK, (cycles == MAX_TICK) ? "PASS" : "FAIL");
          if (cycles == MAX_TICK) pass = pass + 1;
          else fail = fail + 1;
        end
        first_seen = 1'b1;
        cycles = 1;
      end else cycles = cycles + 1;
    end
  end

  initial begin
    reset();
    repeat (100) @(negedge clk);

    if (fail == 0 & pass > 0) $display("Time: %5d | TEST PASS | %0d/%0d", $time, pass, pass + fail);
    else $display("Time: %5d | TEST FAIL | %0d/%0d", $time, pass, pass + fail);
```

```
    $finish();
  end

endmodule
```

tb_value_latch.v

```
`timescale 1ns/1ps
module tb_value_latch (
);

  reg clk = 1'b0;
  always #5 clk = ~clk;

  reg reset_n, pressed_c, pressed_d;
  reg [15:0] sw;

  wire pressed = pressed_c | pressed_d;

  wire dec_hex;
  wire [15:0] sw_reg;

  reg [15:0] exp_sw;
  reg exp_dec_hex;

  integer pass = 0, fail = 0;

  value_latch value_latch_i (clk, reset_n, pressed_c, pressed_d, sw, dec_hex, sw_reg);

  task wait_posedge();
    begin
      @(posedge clk); #1;
    end
  endtask

  task reset();
    begin
      reset_n = 1'b0; pressed_c = 1'b0; pressed_d = 1'b0; sw = 16'b0;
      repeat (1) wait_posedge();

      reset_n = 1'b1;
    end
  endtask

  task check_value(input exp_dec_hex, input [15:0] exp_sw);
    reg res;
    begin
      res = (exp_dec_hex == dec_hex) && (exp_sw == sw_reg);
      $display("Time: %5d | dec_hex=%b exp=%b | sw_reg=%h exp=%h | %s", $time, dec_hex,
exp_dec_hex, sw_reg, exp_sw, res ? "PASS" : "FAIL");
      if (res) pass = pass + 1;
      else fail = fail + 1;
    end
  endtask
```

```
    task drive_values(input press_c, input press_d, input [15:0] sw_val);
        begin
            pressed_c = press_c;
            pressed_d = press_d;
            sw = sw_val;
            wait_posedge();
        end
    endtask

    initial begin
        reset(); check_value(0, 16'b0);
        drive_values(0, 0, 16'h1234); check_value(0, 16'b0);
        drive_values(1, 0, 16'h1234); check_value(1, 16'h1234);
        drive_values(0, 1, 16'h5678); check_value(0, 16'h5678);
        drive_values(0, 0, 16'hABBA); check_value(0, 16'h5678);
        drive_values(1, 1, 16'hFFFF); check_value(0, 16'hFFFF);
        wait_posedge();

        if(fail == 0 & pass > 0) $display("Time: %5d | TEST PASS | %0d/%0d", $time, pass, pass + fail);
        else $display("Time: %5d | TEST FAIL | %0d/%0d", $time, pass, pass + fail);

        $finish();
    end



endmodule
```

tb_btn_hold_1s.v

```
`timescale 1ns/1ps
module tb_btn_hold_1s (
);
    localparam MAX_TICK = 5;
    localparam MAX_COUNT = 2;
    localparam ONE_SEC_CYCLES = MAX_TICK * MAX_COUNT; // The cycles for an actual press
    localparam MIN_LATENCY = 1 + (MAX_COUNT-1)*MAX_TICK; // the fastes 2 ticks (witch is the count for a
pulse) is if we start 1 cycle before the first tick  + the 5 cycles for the next tick
    localparam MAX_LATENCY = ONE_SEC_CYCLES; // the slowes 2 tick we can get is: we get a click just as the
first tick is done (so 5 cycles to first tick) + another 5 cycles to the second tick
    reg clk = 1'b0;
    always #5 clk = ~clk;

    reg reset_n;
    reg tick;
    reg click;
    wire pressed_1s;
    reg expected_pressed;
    integer pass = 0, fail = 0;

    btn_hold_1s #(.MAX_COUNT(MAX_COUNT)) btn_hold_1s_i (clk, reset_n, click, tick, pressed_1s);
```

```
task reset();
  begin
    reset_n = 1'b0; tick = 1'b0; click = 1'b0;
    repeat (2) @(negedge clk);
    reset_n = 1'b1;
  end
endtask

task wait_posedge();
  begin
    @(posedge clk); #1;
  end
endtask

// synchronizer
reg tb_sync0 = 0, tb_sync = 0;
always @(posedge clk) begin
  tb_sync0 <= click;
  tb_sync <= tb_sync0;
end

// Tick gen
integer count = 1;
always @(posedge clk) begin
  if (reset_n) begin
    if (count == MAX_TICK) begin
      tick <= 1'b1;
      count <= 1;
    end else begin
      tick <= 1'b0;
      count <= count + 1;
    end
  end
end

// count the cycles the click is on to make sure the press is only after a correct amount of time
integer cycles_clicked = 0;
always @(posedge clk) cycles_clicked <= tb_sync ? cycles_clicked + 1 : 0;

// count pulses per click and update how many cycles since the click
integer pulses=0;
integer cycles_since_click = 0;
always @(posedge clk) begin
  if (pressed_1s == 1'b1) begin
    pulses <= pulses + 1;
    cycles_since_click <= cycles_clicked + 1;
  end
end

task drive_click(input integer cycles_to_wait);
  begin
    click = 1'b1;
    repeat (cycles_to_wait) wait_posedge(); click = 1'b0;
    repeat (5) wait_posedge();

  end
```

```
    endtask

    // I drive a click for "cycles_to_wait" amount of time.
    // I want to check at the end of the click if my exp_pressed == pressed
    task check_pressed_pulse_once(input integer actual, input integer expected);
        reg res;
        begin
            if (reset_n) begin
                res = (actual == expected);
                $display("Time: %5d | Check if pressed_1s pulsed only once | actual=%0d | expected=%0d | %s",
$time, actual, expected, (res) ? "PASS" : "FAIL");
                if (res) pass = pass + 1;
                else fail = fail + 1;
                pulses = 0;
            end
        end
    endtask

    // I want to check cycles since click to press_1s is correct
    task check_cycles_click_to_press(input integer actual);
        reg res;
        begin
            if (reset_n) begin
                res = (actual >= MIN_LATENCY) && (actual <= MAX_LATENCY);
                $display("Time: %5d | Check press_1s goes high after +-1sec | actual=%0d | min=%0d | max=%0d
| %s", $time, actual, MIN_LATENCY, MAX_LATENCY, res ? "PASS" : "FAIL");
                if (res) pass = pass + 1;
                else fail = fail + 1;
                cycles_since_click = 0;
            end
        end
    endtask

    integer j;
    initial begin
        reset();

        // short press
        drive_click(3);
        check_pressed_pulse_once(pulses, 0);

        //try to press 1 cycle under min_latency
        drive_click(MIN_LATENCY-1); // because sync has delay at start and end we need to when the actual
click ends
        check_pressed_pulse_once(pulses, 0);

        drive_click(5*ONE_SEC_CYCLES);
        check_cycles_click_to_press(cycles_since_click);
        check_pressed_pulse_once(pulses, 1);

        // check all the phases of tick 0 - 5
        for (j = 0; j <= MAX_TICK; j = j + 1) begin
            repeat (j) wait_posedge();
            drive_click(ONE_SEC_CYCLES);
            check_cycles_click_to_press(cycles_since_click);
            check_pressed_pulse_once(pulses, 1);
```

```
        end

        // click on off
        repeat (3) begin
            click = 1'b1; wait_posedge();
            click = 1'b0; wait_posedge();
        end
        drive_click(2*ONE_SEC_CYCLES);
        check_cycles_click_to_press(cycles_since_click);
        check_pressed_pulse_once(pulses, 1);

        if (fail == 0 && pass > 0) $display("Time: %5d | TEST PASS | %0d/%0d", $time, pass, pass + fail);
        else $display("Time: %5d | TEST FAIL | %0d/%0d", $time, pass, pass + fail);

        $finish();
    end

endmodule
```

tb_binary_2_bcd.v

```
`timescale 1ns/1ps
module tb_binary_2_bcd (

);
    reg [15:0] sw;
    reg start;
    wire [19:0] bcd;

    reg clk = 1'b0;
    always #5 clk = ~clk;

    reg reset_n;

    integer pass = 0, fail = 0;

    binary_2_bcd bin2bcd_i (clk, reset_n, sw, start, bcd);

    task reset ();
        begin
            @(negedge clk);
            reset_n = 1'b0;

            repeat (3) @(negedge clk);
            reset_n = 1'b1;
            @(negedge clk);
        end
    endtask

    function [19:0] bin2bcd (input [15:0] bin);
        integer i;
        integer bin_num;
        begin
            if (start) bin2bcd = 20'd0;
```

```
        else begin
            bin_num = bin;
            for (i = 0; i < 5; i = i + 1) begin
                bin2bcd[4*i +: 4] = bin_num % 10;
                bin_num = bin_num / 10;
            end
        end
    end
endfunction

task drive (input [15:0] switches);
    begin
        @(negedge clk);
        sw = switches;
        start = 1'b1;

        @(negedge clk);
        start = 1'b0;

        repeat (16) @(negedge clk);
        check_bcd(bcd, bin2bcd(sw));
    end
endtask

task drive_random();
    begin
        drive($urandom);
    end
endtask

task check_bcd (input [19:0] actual_bcd, input [19:0] expected_bcd);
    reg res;
    begin
        res = (actual_bcd === expected_bcd);
        $display("Time: %5d | Check BCD conversion correct | sw=%b | actual_bcd=%20b |
expected_bcd=%20b | %s", $time, sw, actual_bcd, expected_bcd, (res) ? "PASS" : "FAIL");
        if (res) pass = pass + 1;
        else fail = fail + 1;
    end
endtask

initial begin
    reset();
    // max num is 65535
    // 1. check that start actually puts zeros into bcd
    @(negedge clk); sw = 16'd12345; start = 1'b1; @(posedge clk); #1; check_bcd(bcd, bin2bcd(sw));
    @(negedge clk); start = 1'b0;

    // 2. check max value min value and all the numbers
    drive(16'd0);
    drive(16'd65535);
    drive(16'd12345);
    drive(16'd67890);

    // 3. 20 random checks
    repeat (20) drive_random();
```

```
    // Summary
    if (fail == 0) $display("Time: %5d | TEST PASS | %0d/%0d", $time, pass, pass + fail);
    else $display("Time: %5d | TEST FAIL | %0d/%0d", $time, pass, pass + fail);
    $finish();
  end

endmodule
```

tb_display_ctrl.v

```
`timescale 1ns/1ps
module tb_display_ctrl ();
  reg clk = 0;
  always #5 clk = ~clk;

  reg reset_n;

  reg tick = 1'b0;
  reg dec_hex;
  reg [15:0] hex_val;
  reg [19:0] bcd_val;

  reg [7:0] exp_display_an;

  wire [7:0] display_an;
  wire [6:0] ss_dig;
  wire dp;

  integer pos;

  integer pass = 0, fail = 0;

  task reset();
    begin
      $display("Time: %5d | Applied reset", $time);
      @(negedge clk);
      reset_n = 1'b0;
      exp_display_an = 8'b11111110;

      repeat (3) @(negedge clk);
      reset_n = 1'b1;
      pos = 0;
      display_an_checker("display_an after reset", display_an, exp_display_an);
    end
  endtask

  display_ctrl display_ctrl_i (clk, reset_n, tick, dec_hex, hex_val, bcd_val, display_an, ss_dig, dp);

  function [19:0] bin2bcd (input [15:0] bin_num);
    integer num;
    integer i;
    begin
      i = 0;
```

```
        num = bin_num;
        for (i = 0; i < 5; i = i + 1) begin
          bin2bcd[4*i +: 4] = num % 10;
          num = num / 10;
        end
      end
    endfunction

    task drive (input [15:0] switch);
      begin
        hex_val = switch;
        bcd_val = bin2bcd(switch);
      end
    endtask

    task drive_random ();
      begin
        drive($urandom);
      end
    endtask

    task gen_tick ();
      begin
        if (reset_n) begin
          repeat (2) @(negedge clk);
          tick = 1'b1;

          @(negedge clk); tick = 1'b0;
          // need to move pos here.
          // pos can go from 0 -> 4 (5 places), but keeps incrementing
          // to find place (% 5)
          pos = (pos + 1) % 5;
          display_an_checker("display_an after tick", display_an, exp_an(pos, dec_hex));
          check_ss_dig(ss_dig);
        end
      end
    endtask

    task display_an_checker (input [8*40:1] s, input [7:0] actual, input [7:0] expected);
      reg res;
      begin
        res = (actual === expected);
        $display("Time: %5d | %0s | actual=%b | expected=%b | %0s", $time, s, actual, expected, res ?
"PASS" : "FAIL");
        if (res) pass = pass + 1;
        else fail = fail + 1;
      end
    endtask

    function [7:0] exp_an (input integer pos, input dec_hex);
      begin
        case(pos)
          0: exp_an = 8'b1111_1110;
          1: exp_an = 8'b1111_1101;
          2: exp_an = 8'b1111_1011;
          3: exp_an = 8'b1111_0111;
```

```
            4: exp_an = dec_hex ? 8'b1110_1111 : 8'b1111_1111;
            default: exp_an = 8'bxxxx_xxxx; // checker has ===
        endcase
    end
endfunction

function [4:0] ss2dig (input [6:0] ss_dig);
    begin
        case(ss_dig)
            7'b0000001: ss2dig = 5'h0;
            7'b1001111: ss2dig = 5'h1;
            7'b0010010: ss2dig = 5'h2;
            7'b0000110: ss2dig = 5'h3;
            7'b1001100: ss2dig = 5'h4;
            7'b0100100: ss2dig = 5'h5;
            7'b0100000: ss2dig = 5'h6;
            7'b0001111: ss2dig = 5'h7;
            7'b0000000: ss2dig = 5'h8;
            7'b0000100: ss2dig = 5'h9;
            7'b0001000: ss2dig = 5'ha;
            7'b1100000: ss2dig = 5'hb;
            7'b0110001: ss2dig = 5'hc;
            7'b1000010: ss2dig = 5'hd;
            7'b0110000: ss2dig = 5'he;
            7'b0111000: ss2dig = 5'hf;
            default ss2dig = 5'b10000;
        endcase
    end
endfunction

// One of the outputs of display_ctrl is ss_dig. The current digit in ss code to display.
// I have the correct position of what is supposed to be displayed, what is being displayed,
// a way to convert what is displayed back to a number,
// so I need to check that after i convert back to a number that the number in bcd_val/hex_val at current
// pos is equal to ss_dig
task check_ss_dig (input [6:0] ss_digit);
    reg [4:0] converted_num;
    reg res;
    reg [3:0] expected_num;
    begin
        // 1. convert ss_dig back to a number
        converted_num = ss2dig(ss_digit);

        // 2. Now I have the converted number, need to check if bcd_val in pos is equal to that number
        if (dec_hex | pos != 4) begin
            if (converted_num[4]) begin
                $display("Time: %5d | Error: ss_dig is not a valid digit pattern | ss_dig=%b | FAIL", $time,
ss_digit);
                fail = fail + 1;
            end else begin
                expected_num = dec_hex ? bcd_val[4*pos +: 4] : hex_val[4*pos +: 4];
                res = (expected_num === converted_num[3:0]);
                $display("Time: %5d | Check ss_dig at pos %0d is correct | actual=%0d | expected=%0d | %s",
$time, pos, converted_num[3:0], expected_num, res ? "PASS" : "FAIL");
                if (res) pass = pass + 1;
                else fail = fail + 1;
```

```
        end
      end
    end
endtask

task gen_5_tick ();
    begin
      repeat (5) gen_tick();
    end
endtask


integer j;
initial begin
    dec_hex = 0;
    reset();

    repeat (20) @(negedge clk);
    display_an_checker("display_an stable without tick", display_an, exp_an(pos, dec_hex));

    drive(0);
    gen_5_tick();
    drive(16'h0123);
    gen_5_tick();
    drive(16'h4567);
    gen_5_tick();
    drive(16'h89ef);
    gen_5_tick();
    drive(16'hffff);
    gen_5_tick();
    drive(16'habcd);
    gen_5_tick();
    drive(16'ha2b2);
    gen_5_tick();

    dec_hex = 1'b1;
    drive(16'd0);
    gen_5_tick();
    drive(16'd1234);
    gen_5_tick();
    drive(16'd56789);
    gen_5_tick();
    drive(16'd65535);
    gen_5_tick();
    drive(16'd24680);
    gen_5_tick();

    dec_hex = 1'b1; drive(16'd12345);
    repeat (4) gen_tick();
    dec_hex = 1'b0; #1;
    display_an_checker("AN4 off right after switch to hex", display_an, exp_an(pos, dec_hex));
    @(negedge clk);
    repeat (6) gen_tick();

    dec_hex = 1'b1; drive(16'd11111);
    repeat (2) gen_tick();
```

```
        drive(16'd54321);
        repeat (3) gen_tick();

        for (j=0; j<40; j=j+1) begin
            dec_hex = j[0];
            drive_random();
            gen_5_tick();
        end

        #20;
        if (fail == 0) $display("Time: %5d | TEST PASS | %0d/%0d", $time, pass, pass + fail);
        else $display("Time: %5d | TEST FAIL | %0d/%0d", $time, fail, pass + fail);

        $finish();

    end
endmodule
```

tb_chip_top.v

```
// I pressed X what does the screen show?
`timescale 1ns/1ps
module tb_chip_top (
);
    // To get 100ns button push:
    // f_clk = 100MHz,
    // I want the ticks to have T_tick=10ns so button hold needs to count to 10
    // f_clk * T_tick = 1 -> tick_gen generates a tick every clk cycle.
    // f_tick = 100MHz, T_count = 10ns -> f_tick*T_count = 10 -> btn_hold needs to count to 10 to get 100ns
btn hold.
    // But MAX_TICK cannot be 1 because it breakes tick_gen.v so make it 2.
    // => MAX_COUNT = 5;
    localparam MAX_TICK = 2;
    localparam MAX_COUNT = 5;

    localparam BCD_CALCULATION_TIME = 16;
    localparam SYNC_TIME = 2;

    localparam ROTATION = 5 * MAX_TICK;

    localparam ONE_SEC_CYCLES = MAX_TICK * MAX_COUNT;
    localparam LONG_PRESS = 2 * ONE_SEC_CYCLES;
    localparam SHORT_PRESS = 3;

    reg clk = 1'b0;
    always #5 clk = ~clk;

    reg reset_n;
    reg [15:0] sw;
    reg btnc, btnd;
    wire dp;
    wire [6:0] ss_dig;
    wire [7:0] disp_an;

    integer pass = 0, fail = 0;
```

```
    chip_top #(.MAX_TICK(MAX_TICK), .MAX_COUNT(MAX_COUNT)) chip_top_i(clk, reset_n, sw, btnc, btnd, dp,
ss_dig, disp_an);

    // Always block to check the digit in display i is correct;
    reg [4:0] ss_dig_on_display [7:0];
    reg display_on_off [7:0];
    integer i;
    always @(posedge clk) begin
        for (i = 0; i < 8; i = i + 1) begin // loop throug all the screens and check witch one is on
            if (disp_an[i] == 0) begin // specific display is on
                ss_dig_on_display[i] <= ss2dig(ss_dig);
                display_on_off[i] <= 1;
            end
        end
    end

    integer j;
    task clear_display_vals();
        begin
            for (j = 0 ; j < 8; j = j + 1) begin
                ss_dig_on_display[j] = 5'b11111;
                display_on_off[j] = 0;
            end
        end
    endtask

    function [4:0] ss2dig (input [6:0] ss_dig);
        begin
            case(ss_dig)
                7'b0000001: ss2dig = 5'h0;
                7'b1001111: ss2dig = 5'h1;
                7'b0010010: ss2dig = 5'h2;
                7'b0000110: ss2dig = 5'h3;
                7'b1001100: ss2dig = 5'h4;
                7'b0100100: ss2dig = 5'h5;
                7'b0100000: ss2dig = 5'h6;
                7'b0001111: ss2dig = 5'h7;
                7'b0000000: ss2dig = 5'h8;
                7'b0000100: ss2dig = 5'h9;
                7'b0001000: ss2dig = 5'ha;
                7'b1100000: ss2dig = 5'hb;
                7'b0110001: ss2dig = 5'hc;
                7'b1000010: ss2dig = 5'hd;
                7'b0110000: ss2dig = 5'he;
                7'b0111000: ss2dig = 5'hf;
                default ss2dig = 5'b10000;
            endcase
        end
    endfunction

    function [19:0] bin2bcd (input [15:0] bin_num);
        integer num;
        integer i;
        begin
            i = 0;
```

```
        num = bin_num;
        for (i = 0; i < 5; i = i + 1) begin
            bin2bcd[4*i +: 4] = num % 10;
            num = num / 10;
        end
    end
endfunction

task wait_posedge();
    begin
        @(posedge clk); #1;
    end
endtask


task drive_btn (input btn_c, input btn_d, input integer hold_time);
    begin
        btnc = btn_c; btnd = btn_d;
        repeat (hold_time) wait_posedge();

        btnc = 0; btnd = 0;
        repeat (BCD_CALCULATION_TIME + SYNC_TIME) wait_posedge(); // need to wait for button to sync +
16 cycles of Double Dabble calculation

        clear_display_vals();
        repeat (2 * ROTATION) wait_posedge();
    end
endtask

task drive_sw (input [15:0] sw_val);
    begin
        sw = sw_val;
    end
endtask

task check_display_val(input exp_dec_hex, input [15:0] exp_value);
    reg [19:0] exp_digits;
    reg [7:0] res;
    integer i;
    begin
        res = 8'b0;
        if (reset_n) begin
            exp_digits = (exp_dec_hex) ? bin2bcd(exp_value) : {4'b0, exp_value};
            for (i = 0; i < 8; i = i + 1) begin
                if (i >= 0 && i <=3) begin
                    if (display_on_off[i] && ss_dig_on_display[i] === {1'b0, exp_digits[4*i +: 4]})
                        res[i] = 1'b1;
                end else if (i == 4) begin
                    if (exp_dec_hex) begin
                        if (display_on_off[i] && ss_dig_on_display[i] === {1'b0, exp_digits[4*i +: 4]})
                            res[i] = 1'b1;
                    end else begin
                        if (!display_on_off[i])
                            res[i] = 1'b1;
                    end
                end
```

```
                else begin
                    if (!display_on_off[i])
                        res[i] = 1'b1;
                end

                if (i >= 5 || (i == 4 && !exp_dec_hex)) begin
                    $display("Time: %5d | Check display %0d is off | actual=%s | exp=%s | %s", $time, i, res[i] ?
"OFF" : "ON", "OFF", res[i] ? "PASS" : "FAIL");
                    if (res[i]) pass = pass + 1;
                    else fail = fail + 1;
                end else begin
                    $display ("Time: %5d | Check display %0d | actual=%0h | expected=%0h | %s", $time, i,
ss_dig_on_display[i], exp_digits[4*i +: 4], res[i] ? "PASS" : "FAIL");
                    if (res[i]) pass = pass + 1;
                    else fail = fail + 1;
                end
            end

            $display("Time: %5d | Check dp is off | actual=%b | expected=%b | %s", $time, dp, 1'b1, (dp ===
1'b1) ? "PASS" : "FAIL");
            if (dp === 1'b1) pass = pass + 1;
            else fail = fail + 1;
        end
    end
  endtask

  task reset();
    begin
        reset_n = 1'b0; btnc = 1'b0; btnd = 1'b0; sw = 16'b0;
        repeat (5) wait_posedge();

        reset_n = 1'b1;
        repeat (5) wait_posedge();

        clear_display_vals();
        repeat (2 * ROTATION) wait_posedge();
    end
  endtask

  integer k;
  reg [15:0] rand_val;
  initial begin
    reset();
    check_display_val(0, 16'd0);

    drive_sw(16'd12345);
    drive_btn(1, 0, LONG_PRESS);
    check_display_val(1, 16'd12345);

    drive_sw(16'd777);
    drive_btn(0, 0, LONG_PRESS);
    check_display_val(1, 16'd12345);

    drive_btn(0, 1, LONG_PRESS);
    check_display_val(0, 16'd777);
```

```
        drive_sw(16'd1);
        drive_btn(1, 0, SHORT_PRESS);
        check_display_val(0, 16'd777);

        drive_sw(16'd65535);
        drive_btn(1, 0, LONG_PRESS);
        check_display_val(1, 16'd65535);

        drive_btn(0, 1, LONG_PRESS);
        check_display_val(0, 16'hFFFF);

        drive_sw(16'd0);
        drive_btn(1, 0, LONG_PRESS);
        check_display_val(1, 16'd0);

        drive_sw(16'hABCD);
        drive_btn(1, 1, LONG_PRESS);
        check_display_val(0, 16'hABCD);

        drive_sw(16'd4321);
        drive_btn(1, 0, LONG_PRESS);
        check_display_val(1, 16'd4321);
        reset();
        check_display_val(0, 16'd0);

        for (k = 0; k < 10; k = k + 1) begin
            rand_val = $urandom;
            drive_sw(rand_val);
            if (k % 2) begin
                drive_btn(1, 0, LONG_PRESS);
                check_display_val(1, rand_val);
            end else begin
                drive_btn(0, 1, LONG_PRESS);
                check_display_val(0, rand_val);
            end
        end

        if (fail == 0 && pass > 0) $display("Time: %5d | TEST PASS | %0d/%0d", $time, pass, pass + fail);
        else $display("Time: %5d | TEST FAIL | %0d/%0d", $time, fail, pass + fail);

        $finish();
    end
endmodule
```